Clinic Queue

Application Source Code

Combined project source and documentation

> **Privacy:** Local database, URL, and SMS configuration values have been redacted. This PDF is a reference copy; use the original project files to run the application.

README.md

MD

````
# Clinic Appointment Queue System

Patients book a slot, check in on arrival, and get SMS updates. Staff call the next
patient from a dashboard. A public board shows who is being served.

## Setup (XAMPP/WAMP or any PHP 8.1+ with MySQL)
1. Copy this folder to `htdocs/clinic-queue`.
2. Import `sql/schema.sql` (phpMyAdmin or `mysql -u root -p < sql/schema.sql`).
3. Edit `config/config.php` (DB credentials, `base_url`).
4. Create a staff login (choose your own password): `php tools/create_staff.php admin YOUR_PASSWORD "Clinic Admin"`.
   The schema does not create a default staff account. If you imported an older
   schema that included a placeholder `admin` account, rerun this command using
   the username you want; it replaces that account's password and reactivates it.
   With XAMPP on Windows, run it using `C:\xampp\php\php.exe tools\create_staff.php admin YOUR_PASSWORD "Clinic Admin"`.
5. Open `http://localhost/clinic-queue/public/`

## Record management
Log in as staff and open **Manage records**. Staff can manage patient accounts,
appointments, and queue actions; admins can also manage staff accounts, services,
and notification logs. Patients can update or deactivate their own profile, edit
or cancel booked appointments, and upload their passport photo. Deactivation,
appointment cancellation, queue status changes,
and notification archiving preserve related history instead of deleting records.
Staff accounts are deactivated rather than permanently deleted, and the current
admin cannot deactivate their own account.
Staff can generate an A4-ready patient report from the patient list and print it
using the browser's print dialog.
The staff dashboard also provides a daily served-patients report, with a date
selector and A4 print layout.
When staff mark a patient as served, the system sends a private, expiring
seven-day report link by SMS. Patients can open the link and use the browser's
View report button, then Print / Save as PDF; the confirmation step prevents
automatic SMS link scanners from opening patient data. Set `public_url` in `config/config.php` to the
application's public HTTPS URL before enabling SMS outside local development.

For an existing database, run these migrations once:
```sql
ALTER TABLE patients ADD COLUMN is_active TINYINT(1) NOT NULL DEFAULT 1;
ALTER TABLE patients ADD COLUMN location VARCHAR(150) NULL;
ALTER TABLE patients ADD COLUMN ghana_card_number VARCHAR(15) NULL;
ALTER TABLE services ADD COLUMN is_active TINYINT(1) NOT NULL DEFAULT 1;
ALTER TABLE notifications ADD COLUMN archived_at DATETIME NULL;
ALTER TABLE notifications MODIFY type ENUM('booking_confirmation','queue_update','your_turn','reminder','report_link') NOT NULL;
```
If you already added passport photo support, do not repeat its `passport_photo`
column migration.
Also create the `patient_report_links` table from `sql/schema.sql`.

## Patient passport photos
Patients can upload a JPG, PNG, or WebP photo (up to 5 MB) from their dashboard.
Photos are stored under `storage/patient-photos/`, outside the `public/` directory,
so the web server must be allowed to create and write to that folder and honor the
included `storage/.htaccess` access restriction. The photo endpoint requires the
patient to be logged in and only serves that patient's own image.
For an existing database, run once:
`ALTER TABLE patients ADD COLUMN passport_photo VARCHAR(80) NULL;`
````

```
## SMS
`driver => 'log'` records messages in the `notifications` table (good for dev and demos).
To send real sandbox SMS, create an Africa's Talking sandbox app, then set
`driver => 'africastalking'` and your `at_api_key`.

## Automated tests
`composer install` then `composer test` (PHPUnit, no database needed).

## Structure
- `src/QueueService.php`: check-in, call-next, position and wait logic
- `src/SmsService.php`: SMS sending and logging
- `public/`: pages (patient) and `public/staff/` (staff)
- `sql/schema.sql`: database
- `tests/`: automated tests

## Team workflow (for the "commits from every member" requirement)
One feature branch per person, e.g. `feature/booking`, `feature/staff-dashboard`,
`feature/sms`, `feature/tests`. Merge through pull requests with another member as reviewer.
```

composer.json

JSON

```
{
  "name": "ghana-campus/clinic-queue",
  "description": "Clinic appointment queue system (group project)",
  "require": { "php": ">=8.1" },
  "require-dev": { "phpunit/phpunit": "^10.5" },
  "scripts": { "test": "phpunit --testdox tests" }
}
```

config/config.php

PHP

```php
<?php
// Secrets and machine-specific values are redacted in this source-code PDF.
return [
    'base_url' => '[REDACTED: local URL path]',
    'public_url' => '[REDACTED: public application URL]',
    'db' => [
        'host' => '[REDACTED]',
        'name' => '[REDACTED]',
        'user' => '[REDACTED]',
        'pass' => '[REDACTED]',
    ],
    'sms' => [
        'driver' => '[REDACTED]',
        'at_username' => '[REDACTED]',
        'at_api_key' => '[REDACTED]',
    ],
];
```

public/book.php

PHP

```
<?php
require __DIR__ . '/../src/bootstrap.php';
require_patient();

$services = $db->query("SELECT * FROM services WHERE is_active=1 ORDER BY service_name")->fetchAll();
$slots = [];
for ($t = strtotime('08:00'); $t <= strtotime('16:30'); $t += 1800) $slots[] = date('H:i:s', $t);
$minDate = date('Y-m-d');
$maxDate = date('Y-m-d', strtotime('+7 days'));
$errors = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $serviceId = (int)($_POST['service_id'] ?? 0);
    $date      = $_POST['appointment_date'] ?? '';
    $slot      = $_POST['slot_time'] ?? '';
    $reason    = trim($_POST['reason'] ?? '');

    $svc = null;
    foreach ($services as $s) if ((int)$s['service_id'] === $serviceId) $svc = $s;

    if (!$svc)                                  $errors[] = 'Choose a service.';
    if ($date < $minDate || $date > $maxDate)   $errors[] = 'Pick a date within the next 7 days.';
    if (!in_array($slot, $slots, true))         $errors[] = 'Pick a valid time slot.';

    if (!$errors) {
        $c = $db->prepare("SELECT COUNT(*) FROM appointments WHERE service_id=? AND appointment_date=? AND status<>'cancelled'");
        $c->execute([$serviceId, $date]);
        if ((int)$c->fetchColumn() >= (int)$svc['daily_capacity']) $errors[] = 'That service is fully booked on this date.';
    }

    if (!$errors) {
        try {
            $st = $db->prepare("INSERT INTO appointments (patient_id, service_id, appointment_date, slot_time, reason) VALUES (?,?,?,?,?)");
            $st->execute([$_SESSION['patient_id'], $serviceId, $date, $slot, $reason ?: null]);

            $p = $db->prepare("SELECT phone FROM patients WHERE patient_id=?");
            $p->execute([$_SESSION['patient_id']]);
            $sms->send((int)$_SESSION['patient_id'], $p->fetchColumn(), 'booking_confirmation',
                "Clinic: booked {$svc['service_name']} on $date at " . substr($slot, 0, 5) . ". Check in on arrival.");

            flash('Appointment booked. A confirmation SMS has been sent.', 'success');
            redirect('/dashboard.php');
        } catch (PDOException $e) {
            if ($e->getCode() === '23000') $errors[] = 'You already have a booking at that time.';
            else throw $e;
        }
    }
}
$title = 'Book appointment';
require __DIR__ . '/partials/header.php';
?>
<div class="row justify-content-center"><div class="col-md-6">
  <h3>Book an appointment</h3>
  <?php foreach ($errors as $err): ?><div class="alert alert-danger"><?= e($err) ?></div><?php endforeach; ?>
  <form method="post" class="card card-body">
```

```
    <?= csrf_field() ?>
    <label class="form-label">Service</label>
    <select class="form-select mb-3" name="service_id" required>
      <?php foreach ($services as $s): ?>
        <option value="<?= (int)$s['service_id'] ?>"><?= e($s['service_name']) ?></option>
      <?php endforeach; ?>
    </select>
    <label class="form-label">Date</label>
    <input class="form-control mb-3" type="date" name="appointment_date" min="<?= $minDate ?>" max="<?= $maxDate ?>" value="<?= $minDate ?>" required>
    <label class="form-label">Time slot</label>
    <select class="form-select mb-3" name="slot_time">
      <?php foreach ($slots as $s): ?><option value="<?= $s ?>"><?= substr($s, 0, 5) ?></option><?php endforeach; ?>
    </select>
    <label class="form-label">Reason (optional)</label>
    <input class="form-control mb-3" name="reason" maxlength="255">
    <button class="btn btn-success">Book</button>
  </form>
</div></div>
<?php require __DIR__ . '/partials/footer.php'; ?>
```

public/dashboard.php

PHP

```
<?php
require __DIR__ . '/../src/bootstrap.php';
require_patient();
$pid = (int)$_SESSION['patient_id'];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $apptId = (int)($_POST['appointment_id'] ?? 0);
    try {
        if (($_POST['action'] ?? '') === 'checkin') {
            $queue->checkIn($apptId, $pid);
            flash('You are checked in. See your position below.', 'success');
        } elseif (($_POST['action'] ?? '') === 'cancel') {
            $db->prepare("UPDATE appointments SET status='cancelled' WHERE appointment_id=? AND patient_id=? AND status='booked'")
               ->execute([$apptId, $pid]);
            flash('Appointment cancelled.', 'warning');
        }
    } catch (RuntimeException $e) {
        flash($e->getMessage(), 'danger');
    }
    redirect('/dashboard.php');
}

$st = $db->prepare(
    "SELECT a.*, s.service_name, q.queue_id, q.queue_number, q.status AS queue_status
       FROM appointments a
       JOIN services s ON s.service_id = a.service_id
       LEFT JOIN queue_entries q ON q.appointment_id = a.appointment_id
      WHERE a.patient_id=?
      ORDER BY a.appointment_date DESC, a.slot_time DESC LIMIT 20"
);
$st->execute([$pid]);
$rows = $st->fetchAll();
$photoSt = $db->prepare("SELECT passport_photo FROM patients WHERE patient_id=?");
$photoSt->execute([$pid]);
$passportPhoto = $photoSt->fetchColumn();

$title = 'My appointments';
$refresh = 15;
require __DIR__ . '/partials/header.php';
?>
<div class="d-flex justify-content-between align-items-center mb-3">
  <h3>Hello, <?= e($_SESSION['patient_name'] ?? '') ?></h3>
  <div class="d-flex gap-2">
    <a class="btn btn-outline-success" href="<?= BASE_URL ?>/profile.php">My profile</a>
    <a class="btn btn-success" href="<?= BASE_URL ?>/book.php">Book appointment</a>
  </div>
</div>
<section class="patient-photo-panel d-flex align-items-center gap-3 mb-4">
  <?php if ($passportPhoto): ?>
    <img class="patient-photo" src="<?= BASE_URL ?>/patient-photo.php" alt="Your passport photo">
  <?php else: ?>
    <div class="patient-photo-placeholder" aria-hidden="true">Photo</div>
  <?php endif; ?>
  <div>
    <h5 class="mb-1">Passport photo</h5>
```

```
    <p class="text-muted mb-2">Add or update the photo on your patient profile.</p>
    <a class="btn btn-outline-success btn-sm" href="<?= BASE_URL ?>/passport-photo.php">
      <?= $passportPhoto ? 'Change photo' : 'Upload passport photo' ?>
    </a>
  </div>
</section>
<div class="table-responsive"><table class="table bg-white align-middle">
  <thead><tr><th>Date</th><th>Time</th><th>Service</th><th>Status</th><th>Queue</th><th></th></tr></thead>
  <tbody>
  <?php foreach ($rows as $r): ?>
    <tr>
      <td><?= e($r['appointment_date']) ?></td>
      <td><?= e(substr($r['slot_time'], 0, 5)) ?></td>
      <td><?= e($r['service_name']) ?></td>
      <td><span class="badge text-bg-secondary"><?= e($r['status']) ?></span></td>
      <td>
        <?php if ($r['queue_id'] && $r['queue_status'] === 'waiting'):
            [$ahead, $wait] = $queue->positionOf((int)$r['queue_id']); ?>
          No. <strong><?= (int)$r['queue_number'] ?></strong><br>
          <small><?= $ahead ?> ahead, about <?= $wait ?> min</small>
        <?php elseif ($r['queue_id']): ?>
          No. <?= (int)$r['queue_number'] ?> (<?= e($r['queue_status']) ?>)
        <?php else: ?>-<?php endif; ?>
      </td>
      <td>
        <?php if ($r['status'] === 'booked'): ?>
          <form method="post" class="d-inline"><?= csrf_field() ?>
            <input type="hidden" name="appointment_id" value="<?= (int)$r['appointment_id'] ?>">
            <a class="btn btn-sm btn-outline-success" href="<?= BASE_URL ?>/edit-appointment.php?id=<?= (int)$r['appointment_id'] ?>">Edit</a>
            <?php if ($r['appointment_date'] === date('Y-m-d')): ?>
              <button name="action" value="checkin" class="btn btn-sm btn-success">Check in</button>
            <?php endif; ?>
            <button name="action" value="cancel" class="btn btn-sm btn-outline-danger"
                    onclick="return confirm('Cancel this appointment?')">Cancel</button>
          </form>
        <?php endif; ?>
      </td>
    </tr>
  <?php endforeach; ?>
  <?php if (!$rows): ?><tr><td colspan="6" class="text-center text-muted">No appointments yet.</td></tr><?php endif; ?>
  </tbody>
</table></div>
<?php require __DIR__ . '/partials/footer.php'; ?>
```

public/edit-appointment.php

PHP

```php
<?php
require __DIR__ . '/../src/bootstrap.php';
require_patient();

$patientId = (int)$_SESSION['patient_id'];
$appointmentId = filter_var($_GET['id'] ?? $_POST['appointment_id'] ?? null, FILTER_VALIDATE_INT) ?: 0;
$appointmentSt = $db->prepare(
    "SELECT * FROM appointments WHERE appointment_id=? AND patient_id=? AND status='booked'"
);
$appointmentSt->execute([$appointmentId, $patientId]);
$appointment = $appointmentSt->fetch();
if (!$appointment) {
    http_response_code(404);
    exit('Booked appointment not found.');
}

$services = $db->query('SELECT service_id, service_name, daily_capacity FROM services WHERE is_active=1 ORDER BY service_name')->fetchAll();
$slots = [];
for ($time = strtotime('08:00'); $time <= strtotime('16:30'); $time += 1800) {
    $slots[] = date('H:i', $time);
}
$minDate = date('Y-m-d');
$maxDate = date('Y-m-d', strtotime('+7 days'));
$errors = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $serviceId = (int)($_POST['service_id'] ?? 0);
    $date = $_POST['appointment_date'] ?? '';
    $slot = $_POST['slot_time'] ?? '';
    $reason = trim($_POST['reason'] ?? '');
    $service = null;
    foreach ($services as $candidate) {
        if ((int)$candidate['service_id'] === $serviceId) $service = $candidate;
    }

    if (!$service) $errors[] = 'Choose an available service.';
    if ($date < $minDate || $date > $maxDate) $errors[] = 'Pick a date within the next 7 days.';
    if (!in_array($slot, $slots, true)) $errors[] = 'Pick a valid time slot.';
    if (strlen($reason) > 255) $errors[] = 'Reason must be 255 characters or fewer.';

    if (!$errors) {
        $count = $db->prepare("SELECT COUNT(*) FROM appointments WHERE service_id=? AND appointment_date=? AND status<>'cancelled' AND appointment_id<>?");
        $count->execute([$serviceId, $date, $appointmentId]);
        if ((int)$count->fetchColumn() >= (int)$service['daily_capacity']) {
            $errors[] = 'That service is fully booked on this date.';
        }
    }

    if (!$errors) {
        try {
            $update = $db->prepare(
                "UPDATE appointments SET service_id=?, appointment_date=?, slot_time=?, reason=?
                 WHERE appointment_id=? AND patient_id=? AND status='booked'"
            );
            $update->execute([$serviceId, $date, $slot . ':00', $reason ?: null, $appointmentId, $patientId]);
```

```
            flash('Appointment updated.', 'success');
            redirect('/dashboard.php');
        } catch (PDOException $e) {
            if ($e->getCode() === '23000') $errors[] = 'You already have an appointment at that time.';
            else throw $e;
        }
    }
}

$title = 'Edit appointment';
require __DIR__ . '/partials/header.php';
?>
<div class="row justify-content-center"><div class="col-md-6">
  <h3>Edit appointment</h3>
  <?php foreach ($errors as $error): ?><div class="alert alert-danger"><?= e($error) ?></div><?php endforeach; ?>
  <form method="post" class="card card-body">
    <?= csrf_field() ?>
    <input type="hidden" name="appointment_id" value="<?= $appointmentId ?>">
    <label class="form-label">Service</label>
    <select class="form-select mb-3" name="service_id" required>
      <?php foreach ($services as $service): ?><option value="<?= (int)$service['service_id'] ?>" <?= (int)$appointment['service_id'] === (int)$service['service_id'] ? 'selected' : '' ?>><?=
e($service['service_name']) ?></option><?php endforeach; ?>
    </select>
    <label class="form-label">Date</label>
    <input class="form-control mb-3" type="date" name="appointment_date" min="<?= $minDate ?>" max="<?= $maxDate ?>" value="<?= e($_POST['appointment_date'] ?? $appointment['appointment_date']) ?>"
required>
    <label class="form-label">Time slot</label>
    <select class="form-select mb-3" name="slot_time" required>
      <?php foreach ($slots as $slot): ?><option value="<?= $slot ?>" <?= substr($_POST['slot_time'] ?? $appointment['slot_time'], 0, 5) === $slot ? 'selected' : '' ?>><?= $slot ?></option><?php
endforeach; ?>
    </select>
    <label class="form-label">Reason (optional)</label>
    <input class="form-control mb-3" name="reason" maxlength="255" value="<?= e($_POST['reason'] ?? $appointment['reason'] ?? '') ?>">
    <div class="d-flex gap-2"><button class="btn btn-success">Save changes</button><a class="btn btn-outline-secondary" href="<?= BASE_URL ?>/dashboard.php">Cancel</a></div>
  </form>
</div></div>
<?php require __DIR__ . '/partials/footer.php'; ?>
```

public/index.php

PHP

```
<?php
require __DIR__ . '/../src/bootstrap.php';
$title = 'Clinic Appointment Queue';
require __DIR__ . '/partials/header.php';
?>
<div class="hero-panel p-5 bg-white rounded shadow-sm text-center">
  <h1>Skip the long wait</h1>
  <p class="lead">Book a slot, check in when you arrive, and get an SMS when your turn is near.</p>
  <a class="btn btn-success btn-lg" href="<?= BASE_URL ?>/register.php">Register</a>
  <a class="btn btn-outline-success btn-lg" href="<?= BASE_URL ?>/login.php">Login</a>
  <a class="btn btn-outline-secondary btn-lg" href="<?= BASE_URL ?>/queue.php">See live queue</a>
</div>
<?php require __DIR__ . '/partials/footer.php'; ?>
```

public/login.php

PHP

```
<?php
require __DIR__ . '/../src/bootstrap.php';
$error = null;
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $phone = preg_replace('/\s+/', '', $_POST['phone'] ?? '');
    $st = $db->prepare("SELECT * FROM patients WHERE phone=? AND is_active=1");
    $st->execute([$phone]);
    $p = $st->fetch();
    if ($p && password_verify($_POST['password'] ?? '', $p['password_hash'])) {
        session_regenerate_id(true);
        $_SESSION['patient_id'] = (int)$p['patient_id'];
        $_SESSION['patient_name'] = $p['full_name'];
        redirect('/dashboard.php');
    }
    $error = 'Wrong phone number or password.';
}
$title = 'Patient login';
require __DIR__ . '/partials/header.php';
?>
<div class="row justify-content-center"><div class="col-md-5">
  <h3>Patient login</h3>
  <?php if ($error): ?><div class="alert alert-danger"><?= e($error) ?></div><?php endif; ?>
  <form method="post" class="card card-body">
    <?= csrf_field() ?>
    <label class="form-label">Phone</label>
    <input class="form-control mb-3" name="phone" required>
    <label class="form-label">Password</label>
    <input class="form-control mb-3" type="password" name="password" required>
    <button class="btn btn-success">Login</button>
  </form>
</div></div>
<?php require __DIR__ . '/partials/footer.php'; ?>
```

public/logout.php

PHP

```php
<?php
require __DIR__ . '/../src/bootstrap.php';
$_SESSION = [];
session_destroy();
header('Location: ' . BASE_URL . '/');
exit;
```

public/passport-photo.php

PHP

```php
<?php
require __DIR__ . '/../src/bootstrap.php';
require_patient();

$pid = (int)$_SESSION['patient_id'];
$errors = [];
$maxFileSize = 5 * 1024 * 1024;
$photoDirectory = dirname(__DIR__) . '/storage/patient-photos';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $upload = $_FILES['passport_photo'] ?? null;

    if (!$upload || !isset($upload['error']) || is_array($upload['error'])) {
        $errors[] = 'Choose a passport photo to upload.';
    } elseif ($upload['error'] !== UPLOAD_ERR_OK) {
        $errors[] = match ($upload['error']) {
            UPLOAD_ERR_INI_SIZE, UPLOAD_ERR_FORM_SIZE => 'The photo must be 5 MB or smaller.',
            UPLOAD_ERR_NO_FILE => 'Choose a passport photo to upload.',
            UPLOAD_ERR_PARTIAL => 'The photo upload was incomplete. Please try again.',
            default => 'The photo could not be uploaded. Please try again.',
        };
    } elseif (!isset($upload['tmp_name']) || !is_string($upload['tmp_name']) || !is_uploaded_file($upload['tmp_name'])) {
        $errors[] = 'The uploaded photo could not be verified. Please try again.';
    } elseif ($upload['size'] > $maxFileSize) {
        $errors[] = 'The photo must be 5 MB or smaller.';
    } else {
        $imageInfo = getimagesize($upload['tmp_name']);
        $mimeType = (new finfo(FILEINFO_MIME_TYPE))->file($upload['tmp_name']);
        $imageTypes = [
            'image/jpeg' => ['extension' => 'jpg', 'type' => IMAGETYPE_JPEG],
            'image/png' => ['extension' => 'png', 'type' => IMAGETYPE_PNG],
            'image/webp' => ['extension' => 'webp', 'type' => IMAGETYPE_WEBP],
        ];

        if (!$imageInfo || !isset($imageTypes[$mimeType]) || $imageInfo[2] !== $imageTypes[$mimeType]['type']) {
            $errors[] = 'Use a valid JPG, PNG, or WebP image.';
        } elseif ($imageInfo[0] > 6000 || $imageInfo[1] > 6000 || $imageInfo[0] * $imageInfo[1] > 20000000) {
            $errors[] = 'The image dimensions are too large. Choose a smaller photo.';
        } else {
            try {
                if (!is_dir($photoDirectory) && !mkdir($photoDirectory, 0750, true) && !is_dir($photoDirectory)) {
                    throw new RuntimeException('Could not create the private photo storage directory.');
                }

                $photoName = bin2hex(random_bytes(16)) . '.' . $imageTypes[$mimeType]['extension'];
                $photoPath = $photoDirectory . DIRECTORY_SEPARATOR . $photoName;

                if (!move_uploaded_file($upload['tmp_name'], $photoPath)) {
                    throw new RuntimeException('Could not save the uploaded photo. Please try again.');
                }
            } catch (RuntimeException $e) {
                $errors[] = $e->getMessage();
            }

            if (!$errors) {
```

```
                $previousSt = $db->prepare('SELECT passport_photo FROM patients WHERE patient_id=?');
                $previousSt->execute([$pid]);
                $previousPhoto = $previousSt->fetchColumn();

                try {
                    $update = $db->prepare('UPDATE patients SET passport_photo=? WHERE patient_id=?');
                    $update->execute([$photoName, $pid]);
                } catch (Throwable $e) {
                    if (is_file($photoPath) && !unlink($photoPath)) {
                        error_log('Could not remove an unreferenced patient photo after a database error.');
                    }
                    throw $e;
                }

                if (is_string($previousPhoto) && preg_match('/\A[a-f0-9]{32}\.(?:jpg|png|webp)\z/', $previousPhoto)) {
                    $previousPath = $photoDirectory . DIRECTORY_SEPARATOR . $previousPhoto;
                    if (is_file($previousPath) && !unlink($previousPath)) {
                        error_log('Could not remove a replaced patient photo.');
                    }
                }

                flash('Your passport photo has been uploaded.', 'success');
                redirect('/dashboard.php');
            }
        }
    }
}

$title = 'Upload passport photo';
require __DIR__ . '/partials/header.php';
?>
<div class="row justify-content-center">
  <div class="col-md-6">
    <h3>Passport photo</h3>
    <p class="text-muted">Upload a JPG, PNG, or WebP image up to 5 MB.</p>
    <?php foreach ($errors as $error): ?><div class="alert alert-danger"><?= e($error) ?></div><?php endforeach; ?>
    <form id="passport-photo-form" method="post" enctype="multipart/form-data" class="card card-body">
      <?= csrf_field() ?>
      <input type="hidden" name="MAX_FILE_SIZE" value="<?= $maxFileSize ?>">
      <label class="form-label" for="passport_photo">Choose your photo</label>
      <input class="form-control mb-3" id="passport_photo" type="file" name="passport_photo"
             accept="image/jpeg,image/png,image/webp" required>
      <div class="d-flex gap-2">
        <button class="btn btn-success" type="submit">Upload photo</button>
        <a class="btn btn-outline-secondary" href="<?= BASE_URL ?>/dashboard.php">Cancel</a>
      </div>
    </form>
  </div>
</div>
<?php require __DIR__ . '/partials/footer.php'; ?>
```

public/patient-photo.php

PHP

```php
<?php
require __DIR__ . '/../src/bootstrap.php';
require_patient();

$st = $db->prepare('SELECT passport_photo FROM patients WHERE patient_id=? AND is_active=1');
$st->execute([(int)$_SESSION['patient_id']]);
$photoName = $st->fetchColumn();

if (!is_string($photoName) || !preg_match('/\A[a-f0-9]{32}\.(?:jpg|png|webp)\z/', $photoName)) {
    http_response_code(404);
    exit;
}

$photoPath = dirname(__DIR__) . '/storage/patient-photos/' . $photoName;
if (!is_file($photoPath)) {
    http_response_code(404);
    exit;
}

$mimeType = (new finfo(FILEINFO_MIME_TYPE))->file($photoPath);
if (!in_array($mimeType, ['image/jpeg', 'image/png', 'image/webp'], true)) {
    http_response_code(404);
    exit;
}

header('Content-Type: ' . $mimeType);
header('Content-Disposition: inline');
header('Cache-Control: private, no-store');
header('X-Content-Type-Options: nosniff');
header("Content-Security-Policy: default-src 'none'; sandbox");
readfile($photoPath);
```

public/profile.php

PHP

```php
<?php
require __DIR__ . '/../src/bootstrap.php';
require_patient();

$patientId = (int)$_SESSION['patient_id'];
$getPatient = $db->prepare('SELECT full_name, phone, email, location, ghana_card_number, password_hash FROM patients WHERE patient_id=? AND is_active=1');
$getPatient->execute([$patientId]);
$patient = $getPatient->fetch();
if (!$patient) {
    unset($_SESSION['patient_id'], $_SESSION['patient_name']);
    redirect('/login.php');
}

$errors = [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    if (($_POST['action'] ?? '') === 'deactivate') {
        if (!password_verify($_POST['current_password'] ?? '', $patient['password_hash'])) {
            $errors[] = 'Enter your current password to deactivate your account.';
        } else {
            $db->prepare('UPDATE patients SET is_active=0 WHERE patient_id=?')->execute([$patientId]);
            unset($_SESSION['patient_id'], $_SESSION['patient_name']);
            session_regenerate_id(true);
            flash('Your account has been deactivated. Appointment history has been kept.', 'success');
            redirect('/login.php');
        }
    } elseif (($_POST['action'] ?? '') === 'save') {
        $name = trim($_POST['full_name'] ?? '');
        $phone = preg_replace('/\s+/', '', $_POST['phone'] ?? '');
        $email = trim($_POST['email'] ?? '');
        $location = trim($_POST['location'] ?? '');
        $ghanaCardNumber = strtoupper(preg_replace('/\s+/', '', $_POST['ghana_card_number'] ?? ''));
        $currentPassword = $_POST['current_password'] ?? '';
        $newPassword = $_POST['new_password'] ?? '';
        if ($name === '' || strlen($name) > 100) $errors[] = 'Enter your name (up to 100 characters).';
        if (!preg_match('/^0\d{9}$/', $phone)) $errors[] = 'Enter a 10-digit phone number starting with 0.';
        if ($email !== '' && (!filter_var($email, FILTER_VALIDATE_EMAIL) || strlen($email) > 120)) $errors[] = 'Enter a valid email address.';
        if (strlen($location) > 150) $errors[] = 'Location must be 150 characters or fewer.';
        if ($ghanaCardNumber !== '' && !preg_match('/^GHA-\d{9}-\d$/', $ghanaCardNumber)) $errors[] = 'Enter a valid Ghana Card number (for example, GHA-123456789-0).';
        if ($newPassword !== '' && strlen($newPassword) < 6) $errors[] = 'New password must be at least 6 characters.';
        if ($newPassword !== '' && !password_verify($currentPassword, $patient['password_hash'])) $errors[] = 'Enter your current password to change it.';
        if (!$errors) {
            $sql = 'UPDATE patients SET full_name=?, phone=?, email=?, location=?, ghana_card_number=?' . ($newPassword !== '' ? ', password_hash=?' : '') . ' WHERE patient_id=?';
            $values = [$name, $phone, $email ?: null, $location ?: null, $ghanaCardNumber ?: null];
            if ($newPassword !== '') $values[] = password_hash($newPassword, PASSWORD_DEFAULT);
            $values[] = $patientId;
            try {
                $db->prepare($sql)->execute($values);
                $_SESSION['patient_name'] = $name;
                if ($newPassword !== '') session_regenerate_id(true);
                flash('Your profile has been updated.', 'success');
                redirect('/profile.php');
            } catch (PDOException $e) {
                if ($e->getCode() === '23000') $errors[] = 'That phone number is already registered.';
                else throw $e;
            }
```

```
        }
        $patient['full_name'] = $name;
        $patient['phone'] = $phone;
        $patient['email'] = $email;
        $patient['location'] = $location;
        $patient['ghana_card_number'] = $ghanaCardNumber;
    }
}

$title = 'My profile';
require __DIR__ . '/partials/header.php';
?>
<div class="row justify-content-center"><div class="col-md-6">
  <h3>My profile</h3>
  <?php foreach ($errors as $error): ?><div class="alert alert-danger"><?= e($error) ?></div><?php endforeach; ?>
  <form method="post" class="card card-body mb-4">
    <?= csrf_field() ?>
    <input type="hidden" name="action" value="save">
    <label class="form-label">Full name</label><input class="form-control mb-3" name="full_name" maxlength="100" value="<?= e($patient['full_name']) ?>" required>
    <label class="form-label">Phone</label><input class="form-control mb-3" name="phone" value="<?= e($patient['phone']) ?>" required>
    <label class="form-label">Email (optional)</label><input class="form-control mb-3" type="email" name="email" maxlength="120" value="<?= e($patient['email'] ?? '') ?>">
    <label class="form-label">Location (optional)</label><input class="form-control mb-3" name="location" maxlength="150" value="<?= e($patient['location'] ?? '') ?>">
    <label class="form-label">Ghana Card Number (optional)</label><input class="form-control mb-3" name="ghana_card_number" maxlength="15" placeholder="GHA-123456789-0" value="<?=
e($patient['ghana_card_number'] ?? '') ?>">
    <label class="form-label">Current password (required only to change password)</label><input class="form-control mb-3" type="password" name="current_password">
    <label class="form-label">New password (optional)</label><input class="form-control mb-3" type="password" name="new_password" minlength="6">
    <button class="btn btn-success">Save profile</button>
  </form>
  <form method="post" class="card card-body border-danger" onsubmit="return confirm('Deactivate your account? Your appointment history will be kept, but you will no longer be able to log in.')">
    <?= csrf_field() ?>
    <input type="hidden" name="action" value="deactivate">
    <label class="form-label">Current password to deactivate account</label>
    <input class="form-control mb-3" type="password" name="current_password" required>
    <button class="btn btn-outline-danger">Deactivate my account</button>
  </form>
  <a class="btn btn-link mt-2" href="<?= BASE_URL ?>/dashboard.php">Back to appointments</a>
</div></div>
<?php require __DIR__ . '/partials/footer.php'; ?>
```

public/queue.php

PHP

```
<?php
require __DIR__ . '/../src/bootstrap.php';
$q = $queue->todayQueue();
$title = 'Live queue';
$refresh = 10;
require __DIR__ . '/partials/header.php';
?>
<h3 class="mb-3">Live queue <small class="text-muted fs-6">updates every 10 seconds</small></h3>
<div class="row g-3">
  <div class="col-md-5">
    <div class="card text-center border-success"><div class="card-body">
      <div class="text-muted">Now serving</div>
      <?php if ($q['called']): foreach ($q['called'] as $c): ?>
        <div class="display-3 text-success">#<?= (int)$c['queue_number'] ?></div>
      <?php endforeach; else: ?>
        <div class="display-6 text-muted">-</div>
      <?php endif; ?>
    </div></div>
  </div>
  <div class="col-md-7">
    <div class="card"><div class="card-header">Waiting (<?= count($q['waiting']) ?>)</div>
      <ul class="list-group list-group-flush">
        <?php foreach ($q['waiting'] as $w): ?>
          <li class="list-group-item d-flex justify-content-between">
            <span>#<?= (int)$w['queue_number'] ?> - <?= e($w['service_name']) ?></span>
            <?php if ($w['is_emergency']): ?><span class="badge text-bg-danger">Priority</span><?php endif; ?>
          </li>
        <?php endforeach; ?>
        <?php if (!$q['waiting']): ?><li class="list-group-item text-muted">Nobody waiting.</li><?php endif; ?>
      </ul>
    </div>
  </div>
</div>
<?php require __DIR__ . '/partials/footer.php'; ?>
```

public/register.php

PHP

```php
<?php
require __DIR__ . '/../src/bootstrap.php';
$errors = [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $name  = trim($_POST['full_name'] ?? '');
    $phone = preg_replace('/\s+/', '', $_POST['phone'] ?? '');
    $location = trim($_POST['location'] ?? '');
    $ghanaCardNumber = strtoupper(preg_replace('/\s+/', '', $_POST['ghana_card_number'] ?? ''));
    $pass  = $_POST['password'] ?? '';

    if ($name === '')                          $errors[] = 'Full name is required.';
    if (!preg_match('/^0\d{9}$/', $phone))    $errors[] = 'Enter a 10-digit phone number starting with 0.';
    if (strlen($location) > 150)              $errors[] = 'Location must be 150 characters or fewer.';
    if ($ghanaCardNumber !== '' && !preg_match('/^GHA-\d{9}-\d$/', $ghanaCardNumber)) $errors[] = 'Enter a valid Ghana Card number (for example, GHA-123456789-0).';
    if (strlen($pass) < 6)                     $errors[] = 'Password must be at least 6 characters.';

    if (!$errors) {
        try {
            $st = $db->prepare("INSERT INTO patients (full_name, phone, location, ghana_card_number, password_hash) VALUES (?,?,?,?,?)");
            $st->execute([$name, $phone, $location ?: null, $ghanaCardNumber ?: null, password_hash($pass, PASSWORD_DEFAULT)]);
            session_regenerate_id(true);
            $_SESSION['patient_id'] = (int)$db->lastInsertId();
            $_SESSION['patient_name'] = $name;
            flash('Welcome! Your account is ready.', 'success');
            redirect('/dashboard.php');
        } catch (PDOException $e) {
            if ($e->getCode() === '23000') $errors[] = 'That phone number is already registered.';
            else throw $e;
        }
    }
}
$title = 'Register';
require __DIR__ . '/partials/header.php';
?>
<div class="row justify-content-center"><div class="col-md-5">
  <h3>Create account</h3>
  <?php foreach ($errors as $err): ?><div class="alert alert-danger"><?= e($err) ?></div><?php endforeach; ?>
  <form method="post" class="card card-body">
    <?= csrf_field() ?>
    <label class="form-label">Full name</label>
    <input class="form-control mb-3" name="full_name" value="<?= e($_POST['full_name'] ?? '') ?>" required>
    <label class="form-label">Phone (for SMS)</label>
    <input class="form-control mb-3" name="phone" placeholder="0244123456" value="<?= e($_POST['phone'] ?? '') ?>" required>
    <label class="form-label">Location (optional)</label>
    <input class="form-control mb-3" name="location" maxlength="150" value="<?= e($_POST['location'] ?? '') ?>">
    <label class="form-label">Ghana Card Number (optional)</label>
    <input class="form-control mb-3" name="ghana_card_number" maxlength="15" placeholder="GHA-123456789-0" value="<?= e($_POST['ghana_card_number'] ?? '') ?>">
    <label class="form-label">Password</label>
    <input class="form-control mb-3" type="password" name="password" required>
    <button class="btn btn-success">Register</button>
  </form>
</div></div>
<?php require __DIR__ . '/partials/footer.php'; ?>
```

public/styles.css

CSS

```css
:root {
  --clinic-green: #176b45;
  --clinic-green-dark: #104d34;
  --clinic-green-soft: #eaf4ee;
  --clinic-red: #c74646;
  --clinic-red-soft: #fff0ef;
  --clinic-ink: #26352d;
  --clinic-muted: #718078;
  --clinic-background: #f3f7f4;
  --clinic-border: #dce7df;
}

body.clinic-app {
  min-height: 100vh;
  background:
    radial-gradient(ellipse at 90% 0, rgba(199, 70, 70, 0.07), transparent 28rem),
    var(--clinic-background);
  color: var(--clinic-ink);
}

.clinic-navbar {
  position: relative;
  background: linear-gradient(110deg, var(--clinic-green-dark), var(--clinic-green));
  border-top: 4px solid var(--clinic-red);
  box-shadow: 0 5px 18px rgba(16, 77, 52, 0.14);
}

.clinic-navbar .navbar-brand {
  font-weight: 700;
  letter-spacing: 0.02em;
}

.clinic-navbar .nav-link {
  color: rgba(255, 255, 255, 0.82);
  transition: color 160ms ease;
}

.clinic-navbar .nav-link:hover,
.clinic-navbar .nav-link:focus {
  color: #fff;
}

h1,
h3,
h5 {
  color: var(--clinic-green-dark);
  font-weight: 700;
}

.hero-panel {
  position: relative;
  overflow: hidden;
  border: 1px solid var(--clinic-border);
  border-top: 5px solid var(--clinic-red);
  box-shadow: 0 14px 34px rgba(31, 67, 47, 0.09) !important;
}
```

```css
.hero-panel .lead {
  max-width: 42rem;
  margin: 0.75rem auto 1.5rem;
  color: var(--clinic-muted);
}

.btn-success {
  --bs-btn-bg: var(--clinic-green);
  --bs-btn-border-color: var(--clinic-green);
  --bs-btn-hover-bg: var(--clinic-green-dark);
  --bs-btn-hover-border-color: var(--clinic-green-dark);
  --bs-btn-active-bg: var(--clinic-green-dark);
  --bs-btn-active-border-color: var(--clinic-green-dark);
  --bs-btn-focus-shadow-rgb: 23, 107, 69;
}

.btn-outline-success {
  --bs-btn-color: var(--clinic-green);
  --bs-btn-border-color: var(--clinic-green);
  --bs-btn-hover-bg: var(--clinic-green);
  --bs-btn-hover-border-color: var(--clinic-green);
  --bs-btn-focus-shadow-rgb: 23, 107, 69;
}

.card,
.table,
.form-control,
.form-select {
  border-color: var(--clinic-border);
}

.card {
  border-radius: 0.8rem;
  box-shadow: 0 6px 18px rgba(31, 67, 47, 0.05);
}

.card-header {
  background: var(--clinic-green-soft);
  color: var(--clinic-green-dark);
  font-weight: 600;
}

.card-body .form-control:focus,
.card-body .form-select:focus {
  border-color: var(--clinic-green);
  box-shadow: 0 0 0 0.2rem rgba(23, 107, 69, 0.14);
}

.table {
  overflow: hidden;
  border-radius: 0.65rem;
  box-shadow: 0 5px 16px rgba(31, 67, 47, 0.04);
}

.patient-photo-panel {
  padding: 1rem;
  background: #fff;
  border: 1px solid var(--clinic-border);
  border-left: 4px solid var(--clinic-red);
```

```
  border-radius: 0.8rem;
}

.patient-photo,
.patient-photo-placeholder {
  width: 5rem;
  height: 5rem;
  flex: 0 0 5rem;
  border-radius: 50%;
}

.patient-photo {
  object-fit: cover;
  border: 2px solid var(--clinic-green-soft);
}

.patient-photo-placeholder {
  display: grid;
  place-items: center;
  background: var(--clinic-green-soft);
  color: var(--clinic-green-dark);
  font-size: 0.85rem;
  font-weight: 600;
}

.table thead th {
  background: var(--clinic-green-soft);
  color: var(--clinic-green-dark);
  border-bottom-color: var(--clinic-border);
}

.text-success {
  color: var(--clinic-green) !important;
}

.border-success {
  border-color: var(--clinic-green) !important;
}

.alert-danger {
  color: #812d2d;
  background: var(--clinic-red-soft);
  border-color: #f0c7c4;
}

.alert-success {
  color: var(--clinic-green-dark);
  background: var(--clinic-green-soft);
  border-color: #c9e1d1;
}

.patient-report {
  max-width: 210mm;
  min-height: 297mm;
  margin: 2rem auto;
  padding: 16mm;
  background: #fff;
  color: #26352d;
  box-shadow: 0 8px 30px rgba(31, 67, 47, 0.12);
}
```

```css
.report-actions {
  display: flex;
  justify-content: flex-end;
  gap: 0.5rem;
  margin-bottom: 1.5rem;
}

.report-link-notice {
  padding: 0.5rem;
  background: var(--clinic-green-soft);
  color: var(--clinic-green-dark);
  font-size: 0.85rem;
}

.report-header {
  display: flex;
  justify-content: space-between;
  gap: 1.5rem;
  padding-bottom: 1rem;
  border-bottom: 3px solid var(--clinic-green);
}

.report-kicker {
  margin: 0 0 0.25rem;
  color: var(--clinic-red);
  font-weight: 700;
  letter-spacing: 0.08em;
  text-transform: uppercase;
}

.report-header h1 {
  margin: 0;
  font-size: 1.8rem;
}

.report-subtitle {
  margin: 0.25rem 0 0;
  color: var(--clinic-muted);
}

.report-meta {
  display: grid;
  align-content: center;
  grid-template-columns: auto auto;
  gap: 0.2rem 0.75rem;
  font-size: 0.85rem;
}

.report-section {
  margin-top: 1.5rem;
}

.report-section h2 {
  padding-bottom: 0.4rem;
  color: var(--clinic-green-dark);
  border-bottom: 1px solid var(--clinic-border);
  font-size: 1.1rem;
}
```

```
.report-details {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 0.75rem 1.5rem;
}

.report-details div {
  padding: 0.45rem 0;
  border-bottom: 1px solid #e9efeb;
}

.report-details dt {
  color: var(--clinic-muted);
  font-size: 0.8rem;
}

.report-details dd {
  margin: 0.1rem 0 0;
  font-weight: 600;
}

.report-table {
  width: 100%;
  border-collapse: collapse;
  font-size: 0.82rem;
}

.report-table th,
.report-table td {
  padding: 0.55rem 0.4rem;
  border: 1px solid var(--clinic-border);
  text-align: left;
  vertical-align: top;
  overflow-wrap: anywhere;
}

.report-table th {
  background: var(--clinic-green-soft);
  color: var(--clinic-green-dark);
}

.report-empty {
  color: var(--clinic-muted);
}

.report-footer {
  margin-top: 2rem;
  padding-top: 0.75rem;
  border-top: 1px solid var(--clinic-border);
  color: var(--clinic-muted);
  font-size: 0.75rem;
}

@page {
  size: A4 portrait;
  margin: 12mm;
}

@media print {
  body.clinic-app {
```

```
    min-height: 0;
    background: #fff;
    color: #000;
    font-size: 10pt;
    -webkit-print-color-adjust: exact;
    print-color-adjust: exact;
  }

  .patient-report {
    max-width: none;
    min-height: 0;
    margin: 0;
    padding: 0;
    box-shadow: none;
  }

  .report-actions {
    display: none;
  }

  .report-actions form {
    display: none !important;
  }

  .report-link-notice {
    border: 1px solid var(--clinic-border);
  }

  .report-section,
  .report-details,
  .report-table tr {
    break-inside: avoid;
  }
}

@media (max-width: 575.98px) {
  .clinic-navbar .container {
    flex-wrap: wrap;
  }

  .clinic-navbar .navbar-nav {
    flex-direction: row;
    flex-wrap: wrap;
    gap: 0 0.6rem;
  }

  .hero-panel {
    padding: 2rem 1.25rem !important;
  }

  .patient-report {
    min-height: 0;
    margin: 0.5rem;
    padding: 1rem;
  }

  .report-header {
    flex-direction: column;
  }
```

```
  .report-details {
    grid-template-columns: 1fr;
  }
}
```

public/partials/footer.php

PHP

```
</div>
</body>
</html>
```

public/partials/header.php

PHP

```
<?php /* expects $title; optional $refresh (seconds) */ $f = flash(); ?>
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <?php if (!empty($refresh)): ?><meta http-equiv="refresh" content="<?= (int)$refresh ?>"><?php endif; ?>
  <title><?= e($title ?? 'Clinic Queue') ?></title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
  <link href="<?= BASE_URL ?>/styles.css" rel="stylesheet">
</head>
<body class="clinic-app">
<nav class="navbar navbar-expand navbar-dark clinic-navbar mb-4">
  <div class="container">
    <a class="navbar-brand" href="<?= BASE_URL ?>/">Clinic Queue</a>
    <div class="navbar-nav ms-auto">
      <a class="nav-link" href="<?= BASE_URL ?>/queue.php">Live queue</a>
      <?php if (!empty($_SESSION['patient_id'])): ?>
        <a class="nav-link" href="<?= BASE_URL ?>/dashboard.php">My appointments</a>
        <a class="nav-link" href="<?= BASE_URL ?>/profile.php">My profile</a>
        <a class="nav-link" href="<?= BASE_URL ?>/logout.php">Logout</a>
      <?php elseif (!empty($_SESSION['staff_id'])): ?>
        <a class="nav-link" href="<?= BASE_URL ?>/staff/dashboard.php">Staff dashboard</a>
        <?php if (($_SESSION['staff_role'] ?? '') === 'admin'): ?>
          <a class="nav-link" href="<?= BASE_URL ?>/staff/manage.php">Manage records</a>
        <?php endif; ?>
        <a class="nav-link" href="<?= BASE_URL ?>/logout.php">Logout</a>
      <?php else: ?>
        <a class="nav-link" href="<?= BASE_URL ?>/login.php">Patient login</a>
        <a class="nav-link" href="<?= BASE_URL ?>/staff/login.php">Staff</a>
      <?php endif; ?>
    </div>
  </div>
</nav>
<div class="container">
<?php if ($f): ?><div class="alert alert-<?= e($f['type']) ?>"><?= e($f['msg']) ?></div><?php endif; ?>
```

public/staff/daily-report.php

PHP

```php
<?php
require __DIR__ . '/../../src/bootstrap.php';
require_staff_role($db, ['receptionist', 'nurse', 'admin']);

$date = $_GET['date'] ?? date('Y-m-d');
$parsedDate = DateTimeImmutable::createFromFormat('!Y-m-d', $date);
if (!$parsedDate || $parsedDate->format('Y-m-d') !== $date) {
    http_response_code(400);
    exit('Choose a valid report date.');
}

$st = $db->prepare(
    "SELECT q.queue_number, q.served_at, p.patient_id, p.full_name, p.phone,
            s.service_name, a.appointment_date
     FROM queue_entries q
     JOIN appointments a ON a.appointment_id=q.appointment_id
     JOIN patients p ON p.patient_id=a.patient_id
     JOIN services s ON s.service_id=a.service_id
     WHERE q.status='served' AND DATE(q.served_at)=?
     ORDER BY q.served_at, q.queue_number"
);
$st->execute([$date]);
$servedPatients = $st->fetchAll();

$title = 'Daily served patients report';
header('Cache-Control: private, no-store');
?>
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title><?= e($title) ?> - <?= e($date) ?></title>
  <link href="<?= BASE_URL ?>/styles.css" rel="stylesheet">
</head>
<body class="clinic-app patient-report-page">
  <main class="patient-report">
    <div class="report-actions">
      <form method="get" class="d-flex align-items-center gap-2">
        <label class="form-label mb-0" for="report-date">Report date</label>
        <input class="form-control" id="report-date" type="date" name="date" value="<?= e($date) ?>" required>
        <button class="btn btn-outline-success" type="submit">View</button>
      </form>
      <button class="btn btn-success" type="button" onclick="window.print()">Print A4 report</button>
      <a class="btn btn-outline-secondary" href="<?= BASE_URL ?>/staff/dashboard.php">Back to dashboard</a>
    </div>

    <header class="report-header">
      <div>
        <p class="report-kicker">Clinic Queue</p>
        <h1>Daily Served Patients Report</h1>
        <p class="report-subtitle">Patients served on <?= e($date) ?></p>
      </div>
      <div class="report-meta">
        <strong>Generated</strong>
        <span><?= e(date('Y-m-d H:i')) ?></span>
```

```
        <strong>Total served</strong>
        <span><?= count($servedPatients) ?></span>
      </div>
    </header>

    <section class="report-section">
      <?php if ($servedPatients): ?>
        <table class="report-table">
          <thead>
            <tr>
              <th>#</th>
              <th>Queue no.</th>
              <th>Patient</th>
              <th>Phone</th>
              <th>Service</th>
              <th>Served at</th>
            </tr>
          </thead>
          <tbody>
            <?php foreach ($servedPatients as $index => $patient): ?>
              <tr>
                <td><?= $index + 1 ?></td>
                <td>#<?= (int)$patient['queue_number'] ?></td>
                <td><?= e($patient['full_name']) ?></td>
                <td><?= e($patient['phone']) ?></td>
                <td><?= e($patient['service_name']) ?></td>
                <td><?= e($patient['served_at']) ?></td>
              </tr>
            <?php endforeach; ?>
          </tbody>
        </table>
      <?php else: ?>
        <p class="report-empty">No patients were marked as served on this date.</p>
      <?php endif; ?>
    </section>

    <footer class="report-footer">
      Confidential patient information. Share and store this report securely.
    </footer>
  </main>
</body>
</html>
```

public/staff/dashboard.php

PHP

```
<?php
require __DIR__ . '/../../src/bootstrap.php';
require_staff_role($db, ['receptionist', 'nurse', 'admin']);

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $qid = (int)($_POST['queue_id'] ?? 0);
    switch ($_POST['action'] ?? '') {
        case 'call_next':
            $n = $queue->callNext((int)$_SESSION['staff_id']);
            $n ? flash("Called queue no. {$n['queue_number']}.", 'success') : flash('Nobody is waiting.', 'info');
            break;
        case 'served':
            $reportSent = $queue->markServed($qid);
            flash($reportSent
                ? 'Marked as served. The patient report link was sent by SMS.'
                : 'Marked as served, but the report link SMS could not be sent.', $reportSent ? 'success' : 'warning');
            break;
        case 'no_show':   $queue->markNoShow($qid); flash('Marked as no-show.', 'warning'); break;
        case 'emergency': $queue->setEmergency($qid, true); flash('Moved to priority.', 'danger'); break;
    }
    redirect('/staff/dashboard.php');
}

$q = $queue->todayQueue();
$title = 'Staff dashboard';
$refresh = 20;
require __DIR__ . '/../partials/header.php';
?>
<div class="d-flex justify-content-between align-items-center mb-3">
  <h3>Today's queue</h3>
  <div class="d-flex gap-2">
    <a class="btn btn-outline-secondary" href="<?= BASE_URL ?>/staff/daily-report.php">Today's served report</a>
    <a class="btn btn-outline-success" href="<?= BASE_URL ?>/staff/manage.php">Manage records</a>
    <form method="post"><?= csrf_field() ?>
      <button name="action" value="call_next" class="btn btn-success btn-lg">Call next patient</button>
    </form>
  </div>
</div>

<h5>Currently with staff</h5>
<div class="table-responsive mb-4"><table class="table bg-white align-middle">
  <?php foreach ($q['called'] as $c): ?>
  <tr>
    <td>#<?= (int)$c['queue_number'] ?></td><td><?= e($c['full_name']) ?></td><td><?= e($c['service_name']) ?></td>
    <td class="text-end"><form method="post" class="d-inline"><?= csrf_field() ?>
      <input type="hidden" name="queue_id" value="<?= (int)$c['queue_id'] ?>">
      <button name="action" value="served" class="btn btn-sm btn-success">Served</button>
      <button name="action" value="no_show" class="btn btn-sm btn-outline-secondary">No-show</button>
    </form></td>
  </tr>
  <?php endforeach; ?>
  <?php if (!$q['called']): ?><tr><td class="text-muted">Nobody called yet.</td></tr><?php endif; ?>
</table></div>

<h5>Waiting (<?= count($q['waiting']) ?>)</h5>
```

```
<div class="table-responsive"><table class="table bg-white align-middle">
  <?php foreach ($q['waiting'] as $w): ?>
  <tr class="<?= $w['is_emergency'] ? 'table-danger' : '' ?>">
    <td>#<?= (int)$w['queue_number'] ?></td><td><?= e($w['full_name']) ?></td><td><?= e($w['service_name']) ?></td>
    <td class="text-end"><form method="post" class="d-inline"><?= csrf_field() ?>
      <input type="hidden" name="queue_id" value="<?= (int)$w['queue_id'] ?>">
      <?php if (!$w['is_emergency']): ?>
        <button name="action" value="emergency" class="btn btn-sm btn-outline-danger">Priority</button>
      <?php endif; ?>
      <button name="action" value="no_show" class="btn btn-sm btn-outline-secondary">No-show</button>
    </form></td>
  </tr>
  <?php endforeach; ?>
  <?php if (!$q['waiting']): ?><tr><td class="text-muted">Nobody waiting.</td></tr><?php endif; ?>
</table></div>
<?php require __DIR__ . '/../partials/footer.php'; ?>
```

public/staff/login.php

PHP

```
<?php
require __DIR__ . '/../../src/bootstrap.php';
$error = null;
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $st = $db->prepare("SELECT * FROM staff WHERE username=? AND is_active=1");
    $st->execute([trim($_POST['username'] ?? '')]);
    $s = $st->fetch();
    if ($s && password_verify($_POST['password'] ?? '', $s['password_hash'])) {
        session_regenerate_id(true);
        $_SESSION['staff_id'] = (int)$s['staff_id'];
        $_SESSION['staff_name'] = $s['full_name'];
        $_SESSION['staff_role'] = $s['role'];
        redirect('/staff/dashboard.php');
    }
    $error = 'Wrong username or password.';
}
$title = 'Staff login';
require __DIR__ . '/../partials/header.php';
?>
<div class="row justify-content-center"><div class="col-md-5">
  <h3>Staff login</h3>
  <?php if ($error): ?><div class="alert alert-danger"><?= e($error) ?></div><?php endif; ?>
  <form method="post" class="card card-body">
    <?= csrf_field() ?>
    <label class="form-label">Username</label>
    <input class="form-control mb-3" name="username" required>
    <label class="form-label">Password</label>
    <input class="form-control mb-3" type="password" name="password" required>
    <button class="btn btn-success">Login</button>
  </form>
</div></div>
<?php require __DIR__ . '/../partials/footer.php'; ?>
```

public/staff/manage.php

PHP

```php
<?php
require __DIR__ . '/../../src/bootstrap.php';
$staff = require_staff_role($db, ['receptionist', 'nurse', 'admin']);
$isAdmin = $staff['role'] === 'admin';
$sections = [
    'patients' => 'Patients',
    'appointments' => 'Appointments',
    'queue' => 'Queue',
];
if ($isAdmin) {
    $sections += [
        'staff' => 'Staff',
        'services' => 'Services',
        'notifications' => 'Notifications',
    ];
}
$section = $_GET['section'] ?? 'patients';
if (!isset($sections[$section])) {
    http_response_code(403);
    exit('You do not have permission to manage this record type.');
}
$errors = [];
$today = date('Y-m-d');

$dateIsValid = static function (string $value): bool {
    $date = DateTimeImmutable::createFromFormat('!Y-m-d', $value);
    return $date && $date->format('Y-m-d') === $value;
};

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $action = $_POST['action'] ?? '';
    $id = filter_var($_POST['id'] ?? null, FILTER_VALIDATE_INT) ?: 0;

    try {
        switch ($section) {
            case 'patients':
                if ($action === 'deactivate' && $id > 0) {
                    $db->prepare('UPDATE patients SET is_active=0 WHERE patient_id=?')->execute([$id]);
                    flash('Patient account deactivated. Appointment history was kept.', 'warning');
                } elseif ($action === 'save') {
                    $name = trim($_POST['full_name'] ?? '');
                    $phone = preg_replace('/\s+/', '', $_POST['phone'] ?? '');
                    $email = trim($_POST['email'] ?? '');
                    $location = trim($_POST['location'] ?? '');
                    $ghanaCardNumber = strtoupper(preg_replace('/\s+/', '', $_POST['ghana_card_number'] ?? ''));
                    $password = $_POST['password'] ?? '';
                    if ($name === '' || strlen($name) > 100) $errors[] = 'Enter a patient name (up to 100 characters).';
                    if (!preg_match('/^0\d{9}$/', $phone)) $errors[] = 'Enter a 10-digit phone number starting with 0.';
                    if ($email !== '' && (!filter_var($email, FILTER_VALIDATE_EMAIL) || strlen($email) > 120)) $errors[] = 'Enter a valid email address (up to 120 characters).';
                    if (strlen($location) > 150) $errors[] = 'Location must be 150 characters or fewer.';
                    if ($ghanaCardNumber !== '' && !preg_match('/^GHA-\d{9}-\d$/', $ghanaCardNumber)) $errors[] = 'Enter a valid Ghana Card number (for example, GHA-123456789-0).';
                    if (!$id && strlen($password) < 6) $errors[] = 'New patient passwords must be at least 6 characters.';
                    if ($password !== '' && strlen($password) < 6) $errors[] = 'Passwords must be at least 6 characters.';
                    if (!$errors) {
                        if ($id) {
```

```
                    $sql = 'UPDATE patients SET full_name=?, phone=?, email=?, location=?, ghana_card_number=?, is_active=1' . ($password !== '' ? ', password_hash=?' : '') . ' WHERE patient_id=?';
                    $values = [$name, $phone, $email ?: null, $location ?: null, $ghanaCardNumber ?: null];
                    if ($password !== '') $values[] = password_hash($password, PASSWORD_DEFAULT);
                    $values[] = $id;
                    $db->prepare($sql)->execute($values);
                    flash('Patient record updated and activated.', 'success');
                } else {
                    $db->prepare('INSERT INTO patients (full_name, phone, email, location, ghana_card_number, password_hash) VALUES (?,?,?,?,?,?)')
                        ->execute([$name, $phone, $email ?: null, $location ?: null, $ghanaCardNumber ?: null, password_hash($password, PASSWORD_DEFAULT)]);
                    flash('Patient account created.', 'success');
                }
            }
        }
        break;

    case 'appointments':
        if ($action === 'cancel' && $id > 0) {
            $db->prepare("UPDATE appointments SET status='cancelled' WHERE appointment_id=? AND status='booked'")->execute([$id]);
            flash('Appointment cancelled; history was kept.', 'warning');
        } elseif ($action === 'save') {
            $patientId = (int)($_POST['patient_id'] ?? 0);
            $serviceId = (int)($_POST['service_id'] ?? 0);
            $date = trim($_POST['appointment_date'] ?? '');
            $time = trim($_POST['slot_time'] ?? '');
            $reason = trim($_POST['reason'] ?? '');
            $patientSt = $db->prepare('SELECT COUNT(*) FROM patients WHERE patient_id=? AND is_active=1');
            $patientSt->execute([$patientId]);
            $serviceSt = $db->prepare('SELECT daily_capacity FROM services WHERE service_id=? AND is_active=1');
            $serviceSt->execute([$serviceId]);
            $capacity = $serviceSt->fetchColumn();
            if (!$patientId || !(int)$patientSt->fetchColumn()) $errors[] = 'Choose an active patient.';
            if ($capacity === false) $errors[] = 'Choose an active service.';
            if (!$dateIsValid($date)) $errors[] = 'Choose a valid appointment date.';
            elseif ($date < $today) $errors[] = 'Choose today or a future appointment date.';
            if (!preg_match('/^(?:0[8-9]|1[0-6]):[0-5][0-9]$/', $time) || !in_array(substr($time, 3, 2), ['00', '30'], true) || ($time > '16:30')) $errors[] = 'Choose a valid half-hour slot from
08:00 to 16:30.';
            if (strlen($reason) > 255) $errors[] = 'Reason must be 255 characters or fewer.';
            if (!$errors) {
                $capSt = $db->prepare("SELECT COUNT(*) FROM appointments WHERE service_id=? AND appointment_date=? AND status<>'cancelled' AND appointment_id<>?");
                $capSt->execute([$serviceId, $date, $id]);
                if ((int)$capSt->fetchColumn() >= (int)$capacity) $errors[] = 'That service is fully booked on this date.';
            }
            if (!$errors) {
                if ($id) {
                    $update = $db->prepare("UPDATE appointments SET patient_id=?, service_id=?, appointment_date=?, slot_time=?, reason=? WHERE appointment_id=? AND status='booked'");
                    $update->execute([$patientId, $serviceId, $date, $time . ':00', $reason ?: null, $id]);
                    if ($update->rowCount() === 0) {
                        $check = $db->prepare("SELECT COUNT(*) FROM appointments WHERE appointment_id=? AND status='booked'");
                        $check->execute([$id]);
                        if (!(int)$check->fetchColumn()) $errors[] = 'Only booked appointments can be edited.';
                    }
                    if (!$errors) flash('Appointment updated.', 'success');
                } else {
                    $db->prepare("INSERT INTO appointments (patient_id, service_id, appointment_date, slot_time, reason) VALUES (?,?,?,?,?)")
                        ->execute([$patientId, $serviceId, $date, $time . ':00', $reason ?: null]);
                    flash('Appointment created.', 'success');
                }
            }
        }
        break;
```

```
case 'staff':
    if ($action === 'deactivate' && $id > 0) {
        if ($id === (int)$_SESSION['staff_id']) {
            $errors[] = 'You cannot deactivate your own account.';
        } else {
            $target = $db->prepare("SELECT role FROM staff WHERE staff_id=? AND is_active=1");
            $target->execute([$id]);
            $targetRole = $target->fetchColumn();
            if ($targetRole === 'admin') {
                $countAdmins = $db->query("SELECT COUNT(*) FROM staff WHERE role='admin' AND is_active=1")->fetchColumn();
                if ((int)$countAdmins <= 1) {
                    $errors[] = 'The last active admin account cannot be deactivated.';
                }
            }
            if (!$errors) {
                $db->prepare('UPDATE staff SET is_active=0 WHERE staff_id=?')->execute([$id]);
                flash('Staff account deactivated; queue history was kept.', 'warning');
            }
        }
    } elseif ($action === 'save') {
        $name = trim($_POST['full_name'] ?? '');
        $username = trim($_POST['username'] ?? '');
        $role = $_POST['role'] ?? '';
        $password = $_POST['password'] ?? '';
        if ($name === '' || strlen($name) > 100) $errors[] = 'Enter a staff name (up to 100 characters).';
        if ($username === '' || strlen($username) > 50) $errors[] = 'Enter a username (up to 50 characters).';
        if (!in_array($role, ['receptionist', 'nurse', 'admin'], true)) $errors[] = 'Choose a valid staff role.';
        if ($id === (int)$_SESSION['staff_id'] && $role !== 'admin') $errors[] = 'You cannot remove your own admin role.';
        if (!$id && strlen($password) < 6) $errors[] = 'New staff passwords must be at least 6 characters.';
        if ($password !== '' && strlen($password) < 6) $errors[] = 'Passwords must be at least 6 characters.';
        if (!$errors) {
            if ($id) {
                $sql = 'UPDATE staff SET full_name=?, username=?, role=?, is_active=1' . ($password !== '' ? ', password_hash=?' : '') . ' WHERE staff_id=?';
                $values = [$name, $username, $role];
                if ($password !== '') $values[] = password_hash($password, PASSWORD_DEFAULT);
                $values[] = $id;
                $db->prepare($sql)->execute($values);
                flash('Staff record updated and activated.', 'success');
            } else {
                $db->prepare('INSERT INTO staff (full_name, username, password_hash, role) VALUES (?,?,?,?)')
                    ->execute([$name, $username, password_hash($password, PASSWORD_DEFAULT), $role]);
                flash('Staff account created.', 'success');
            }
        }
    }
    break;

case 'services':
    if ($action === 'deactivate' && $id > 0) {
        $db->prepare('UPDATE services SET is_active=0 WHERE service_id=?')->execute([$id]);
        flash('Service deactivated; existing appointments were kept.', 'warning');
    } elseif ($action === 'save') {
        $name = trim($_POST['service_name'] ?? '');
        $minutes = filter_var($_POST['avg_minutes'] ?? null, FILTER_VALIDATE_INT);
        $capacity = filter_var($_POST['daily_capacity'] ?? null, FILTER_VALIDATE_INT);
        if ($name === '' || strlen($name) > 80) $errors[] = 'Enter a service name (up to 80 characters).';
        if ($minutes === false || $minutes < 1 || $minutes > 600) $errors[] = 'Average duration must be between 1 and 600 minutes.';
        if ($capacity === false || $capacity < 1 || $capacity > 5000) $errors[] = 'Daily capacity must be between 1 and 5000.';
        if (!$errors) {
```

```
            if ($id) {
                $db->prepare('UPDATE services SET service_name=?, avg_minutes=?, daily_capacity=?, is_active=1 WHERE service_id=?')
                    ->execute([$name, $minutes, $capacity, $id]);
                flash('Service updated and activated.', 'success');
            } else {
                $db->prepare('INSERT INTO services (service_name, avg_minutes, daily_capacity) VALUES (?,?,?)')
                    ->execute([$name, $minutes, $capacity]);
                flash('Service created.', 'success');
            }
        }
    }
    break;

case 'queue':
    $queueId = (int)($_POST['queue_id'] ?? 0);
    if ($action === 'check_in') {
        $appointmentId = (int)($_POST['appointment_id'] ?? 0);
        $apptSt = $db->prepare("SELECT patient_id FROM appointments WHERE appointment_id=? AND appointment_date=? AND status='booked'");
        $apptSt->execute([$appointmentId, $today]);
        $patientId = $apptSt->fetchColumn();
        if (!$patientId) {
            $errors[] = 'Choose a booked appointment scheduled for today.';
        } else {
            $queue->checkIn($appointmentId, (int)$patientId);
            flash('Patient checked in to the queue.', 'success');
        }
    } elseif ($action === 'call_next') {
        $called = $queue->callNext((int)$_SESSION['staff_id']);
        flash($called ? 'Next patient called.' : 'Nobody is waiting.', $called ? 'success' : 'info');
    } elseif ($action === 'served' && $queueId) {
        $queueSt = $db->prepare("SELECT COUNT(*) FROM queue_entries WHERE queue_id=? AND status IN ('waiting','called')");
        $queueSt->execute([$queueId]);
        if (!(int)$queueSt->fetchColumn()) $errors[] = 'Only waiting or called patients can be marked as served.';
        else {
            $reportSent = $queue->markServed($queueId);
            flash($reportSent
                ? 'Patient marked as served. The report link was sent by SMS.'
                : 'Patient marked as served, but the report link SMS could not be sent.', $reportSent ? 'success' : 'warning');
        }
    } elseif ($action === 'no_show' && $queueId) {
        $queueSt = $db->prepare("SELECT COUNT(*) FROM queue_entries WHERE queue_id=? AND status IN ('waiting','called')");
        $queueSt->execute([$queueId]);
        if (!(int)$queueSt->fetchColumn()) $errors[] = 'Only waiting or called patients can be marked as no-show.';
        else {
            $queue->markNoShow($queueId);
            flash('Patient marked as no-show; queue history was kept.', 'warning');
        }
    } elseif ($action === 'priority' && $queueId) {
        $queueSt = $db->prepare("SELECT COUNT(*) FROM queue_entries WHERE queue_id=? AND status='waiting' AND is_emergency=0");
        $queueSt->execute([$queueId]);
        if (!(int)$queueSt->fetchColumn()) $errors[] = 'Only non-priority waiting patients can be moved to priority.';
        else {
            $queue->setEmergency($queueId, true);
            flash('Patient moved to priority.', 'warning');
        }
    }
    break;

case 'notifications':
    if ($action === 'archive' && $id > 0) {
```

```
                    $db->prepare('UPDATE notifications SET archived_at=NOW() WHERE notification_id=? AND archived_at IS NULL')->execute([$id]);
                    flash('Notification archived; delivery history was kept.', 'warning');
                } elseif ($action === 'save') {
                    $patientId = (int)($_POST['patient_id'] ?? 0);
                    $queueId = (int)($_POST['queue_id'] ?? 0) ?: null;
                    $type = $_POST['type'] ?? '';
                    $message = trim($_POST['message'] ?? '');
                    $patientSt = $db->prepare('SELECT COUNT(*) FROM patients WHERE patient_id=?');
                    $patientSt->execute([$patientId]);
                    $validQueue = true;
                    if ($queueId) {
                        $queueSt = $db->prepare(
                            'SELECT COUNT(*) FROM queue_entries q
                             JOIN appointments a ON a.appointment_id=q.appointment_id
                             WHERE q.queue_id=? AND a.patient_id=?'
                        );
                        $queueSt->execute([$queueId, $patientId]);
                        $validQueue = (int)$queueSt->fetchColumn() > 0;
                    }
                    if (!$patientId || !(int)$patientSt->fetchColumn()) $errors[] = 'Choose a patient.';
                    if (!$validQueue) $errors[] = 'Choose a valid queue entry or leave the queue blank.';
                    if (!in_array($type, ['booking_confirmation', 'queue_update', 'your_turn', 'reminder', 'report_link'], true)) $errors[] = 'Choose a valid notification type.';
                    if ($message === '' || strlen($message) > 320) $errors[] = 'Message must be between 1 and 320 characters.';
                    if (!$errors && $id) {
                        $db->prepare("UPDATE notifications SET patient_id=?, queue_id=?, type=?, message=? WHERE notification_id=? AND archived_at IS NULL")
                            ->execute([$patientId, $queueId, $type, $message, $id]);
                        flash('Notification log updated.', 'success');
                    } elseif (!$errors) {
                        $db->prepare("INSERT INTO notifications (patient_id, queue_id, type, message, delivery_status) VALUES (?,?,?,?,'queued')")
                            ->execute([$patientId, $queueId, $type, $message]);
                        flash('Notification log entry created (queued; no SMS was sent).', 'success');
                    }
                }
                break;
        }
    } catch (PDOException $e) {
        if ($e->getCode() === '23000') {
            $errors[] = 'This change conflicts with an existing record or referenced history.';
        } else {
            throw $e;
        }
    } catch (RuntimeException $e) {
        $errors[] = $e->getMessage();
    }

    if (!$errors) {
        redirect('/staff/manage.php?section=' . rawurlencode($section));
    }
}

$editId = filter_var($_GET['edit'] ?? null, FILTER_VALIDATE_INT) ?: 0;
$record = null;
$rows = [];
$patients = [];
$services = [];
$appointments = [];
$queueEntries = [];

if (in_array($section, ['patients', 'appointments', 'notifications'], true)) {
    $patientFilter = $section === 'appointments' ? ' WHERE is_active=1' : '';
```

```
    $patients = $db->query('SELECT patient_id, full_name, phone FROM patients' . $patientFilter . ' ORDER BY full_name')->fetchAll();
}
if (in_array($section, ['appointments'], true)) {
    $services = $db->query('SELECT service_id, service_name FROM services WHERE is_active=1 ORDER BY service_name')->fetchAll();
}
if ($section === 'queue') {
    $appointments = $db->query(
        "SELECT a.appointment_id, p.full_name, s.service_name FROM appointments a
         JOIN patients p ON p.patient_id=a.patient_id JOIN services s ON s.service_id=a.service_id
         WHERE a.appointment_date=CURDATE() AND a.status='booked' ORDER BY a.slot_time"
    )->fetchAll();
}
if ($section === 'notifications') {
    $queueEntries = $db->query('SELECT queue_id, queue_number, queue_date FROM queue_entries ORDER BY queue_date DESC, queue_number DESC LIMIT 200')->fetchAll();
}

if ($editId) {
    $tables = [
        'patients' => ['SELECT patient_id AS id, full_name, phone, email, location, ghana_card_number, is_active FROM patients WHERE patient_id=?', true],
        'appointments' => ["SELECT appointment_id AS id, patient_id, service_id, appointment_date, TIME_FORMAT(slot_time,'%H:%i') AS slot_time, reason FROM appointments WHERE appointment_id=? AND
status='booked'", false],
        'staff' => ['SELECT staff_id AS id, full_name, username, role, is_active FROM staff WHERE staff_id=?', false],
        'services' => ['SELECT service_id AS id, service_name, avg_minutes, daily_capacity, is_active FROM services WHERE service_id=?', false],
        'notifications' => ['SELECT notification_id AS id, patient_id, queue_id, type, message FROM notifications WHERE notification_id=? AND archived_at IS NULL', false],
    ];
    if (isset($tables[$section])) {
        [$sql] = $tables[$section];
        $editSt = $db->prepare($sql);
        $editSt->execute([$editId]);
        $record = $editSt->fetch() ?: null;
    }
}

switch ($section) {
    case 'patients':
        $rows = $db->query('SELECT patient_id, full_name, phone, email, is_active FROM patients ORDER BY patient_id DESC LIMIT 200')->fetchAll();
        break;
    case 'appointments':
        $rows = $db->query(
            'SELECT a.appointment_id, a.appointment_date, a.slot_time, a.reason, a.status, p.full_name, s.service_name
             FROM appointments a JOIN patients p ON p.patient_id=a.patient_id JOIN services s ON s.service_id=a.service_id
             ORDER BY a.appointment_date DESC, a.slot_time DESC LIMIT 200'
        )->fetchAll();
        break;
    case 'staff':
        $rows = $db->query('SELECT staff_id, full_name, username, role, is_active FROM staff ORDER BY staff_id')->fetchAll();
        break;
    case 'services':
        $rows = $db->query('SELECT service_id, service_name, avg_minutes, daily_capacity, is_active FROM services ORDER BY service_name')->fetchAll();
        break;
    case 'queue':
        $rows = $db->query(
            'SELECT q.queue_id, q.queue_number, q.queue_date, q.status, q.is_emergency, p.full_name, s.service_name
             FROM queue_entries q JOIN appointments a ON a.appointment_id=q.appointment_id
             JOIN patients p ON p.patient_id=a.patient_id JOIN services s ON s.service_id=a.service_id
             ORDER BY q.queue_date DESC, q.queue_number DESC LIMIT 200'
        )->fetchAll();
        break;
    case 'notifications':
        $rows = $db->query(
```

```
            'SELECT n.notification_id, n.type, n.message, n.delivery_status, n.created_at, p.full_name
             FROM notifications n JOIN patients p ON p.patient_id=n.patient_id
             WHERE n.archived_at IS NULL ORDER BY n.notification_id DESC LIMIT 200'
        )->fetchAll();
        break;
}

$title = 'Manage ' . $sections[$section];
require __DIR__ . '/../partials/header.php';
?>
<div class="d-flex flex-wrap justify-content-between align-items-center gap-2 mb-3">
  <h3 class="mb-0">Manage records</h3>
  <div class="btn-group flex-wrap" role="group" aria-label="Record type">
    <?php foreach ($sections as $key => $label): ?>
      <a class="btn btn-sm <?= $section === $key ? 'btn-success' : 'btn-outline-success' ?>"
         href="<?= BASE_URL ?>/staff/manage.php?section=<?= e($key) ?>"><?= e($label) ?></a>
    <?php endforeach; ?>
  </div>
</div>
<h5 class="mb-3"><?= e($sections[$section]) ?></h5>
<?php foreach ($errors as $error): ?><div class="alert alert-danger"><?= e($error) ?></div><?php endforeach; ?>

<div class="card card-body mb-4">
  <h6><?= $record ? 'Edit record' : 'Create record' ?></h6>
  <?php if ($section === 'queue'): ?>
    <form method="post" class="mb-2">
      <?= csrf_field() ?>
      <label class="form-label">Check in a booked appointment for today</label>
      <select class="form-select mb-2" name="appointment_id" required><option value="">Choose appointment</option><?php foreach ($appointments as $a): ?><option value="<?= (int)$a['appointment_id'] ?>"><?=
e($a['full_name'] . ' — ' . $a['service_name']) ?></option><?php endforeach; ?></select>
      <button class="btn btn-success" name="action" value="check_in">Check in</button>
    </form>
    <form method="post">
      <?= csrf_field() ?>
      <button class="btn btn-outline-success" name="action" value="call_next">Call next patient</button>
    </form>
  <?php else: ?>
  <form method="post">
    <?= csrf_field() ?>
    <input type="hidden" name="action" value="save">
    <?php if ($record): ?><input type="hidden" name="id" value="<?= (int)$record['id'] ?>"><?php endif; ?>
    <?php if ($section === 'patients'): ?>
      <div class="row g-2">
        <div class="col-md-4"><label class="form-label">Full name</label><input class="form-control" name="full_name" maxlength="100" value="<?= e($record['full_name'] ?? '') ?>" required></div>
        <div class="col-md-4"><label class="form-label">Phone</label><input class="form-control" name="phone" value="<?= e($record['phone'] ?? '') ?>" placeholder="0244123456" required></div>
        <div class="col-md-4"><label class="form-label">Email (optional)</label><input class="form-control" type="email" name="email" maxlength="120" value="<?= e($record['email'] ?? '') ?>"></div>
      </div>
      <div class="row g-2 mt-1">
        <div class="col-md-6"><label class="form-label">Location (optional)</label><input class="form-control" name="location" maxlength="150" value="<?= e($record['location'] ?? '') ?>"></div>
        <div class="col-md-6"><label class="form-label">Ghana Card Number (optional)</label><input class="form-control" name="ghana_card_number" maxlength="15" placeholder="GHA-123456789-0" value="<?=
e($record['ghana_card_number'] ?? '') ?>"></div>
      </div>
      <label class="form-label mt-2">Password <?= $record ? '(leave blank to keep current)' : '' ?></label><input class="form-control" type="password" name="password" minlength="6" <?= $record ? '' :
'required' ?>>
    <?php elseif ($section === 'appointments'): ?>
      <div class="row g-2">
        <div class="col-md-4"><label class="form-label">Patient</label><select class="form-select" name="patient_id" required><?php foreach ($patients as $p): ?><option value="<?= (int)$p['patient_id'] ?>"
<?= (int)($record['patient_id'] ?? 0) === (int)$p['patient_id'] ? 'selected' : '' ?>><?= e($p['full_name'] . ' (' . $p['phone'] . ')') ?></option><?php endforeach; ?></select></div>
        <div class="col-md-4"><label class="form-label">Service</label><select class="form-select" name="service_id" required><?php foreach ($services as $s): ?><option value="<?= (int)$s['service_id'] ?>"
<?= (int)($record['service_id'] ?? 0) === (int)$s['service_id'] ? 'selected' : '' ?>><?= e($s['service_name']) ?></option><?php endforeach; ?></select></div>
```

```
        <div class="col-md-2"><label class="form-label">Date</label><input class="form-control" type="date" name="appointment_date" min="<?= $today ?>" value="<?= e($record['appointment_date'] ?? $today) ?
>" required></div>
        <div class="col-md-2"><label class="form-label">Time</label><input class="form-control" type="time" step="1800" min="08:00" max="16:30" name="slot_time" value="<?= e($record['slot_time'] ??
'08:00') ?>" required></div>
      </div>
      <label class="form-label mt-2">Reason (optional)</label><input class="form-control" name="reason" maxlength="255" value="<?= e($record['reason'] ?? '') ?>">
      <?php if ($record): ?><small class="text-muted">Only booked appointments can be edited. Cancellation preserves history.</small><?php endif; ?>
    <?php elseif ($section === 'staff'): ?>
      <div class="row g-2">
        <div class="col-md-4"><label class="form-label">Full name</label><input class="form-control" name="full_name" maxlength="100" value="<?= e($record['full_name'] ?? '') ?>" required></div>
        <div class="col-md-4"><label class="form-label">Username</label><input class="form-control" name="username" maxlength="50" value="<?= e($record['username'] ?? '') ?>" required></div>
        <div class="col-md-4"><label class="form-label">Role</label><select class="form-select" name="role"><?php foreach (['receptionist', 'nurse', 'admin'] as $role): ?><option value="<?= $role ?>" <?=
($record['role'] ?? 'receptionist') === $role ? 'selected' : '' ?>><?= e(ucfirst($role)) ?></option><?php endforeach; ?></select></div>
      </div>
      <label class="form-label mt-2">Password <?= $record ? '(leave blank to keep current)' : '' ?></label><input class="form-control" type="password" name="password" minlength="6" <?= $record ? '' :
'required' ?>>
    <?php elseif ($section === 'services'): ?>
      <div class="row g-2">
        <div class="col-md-6"><label class="form-label">Service name</label><input class="form-control" name="service_name" maxlength="80" value="<?= e($record['service_name'] ?? '') ?>" required></div>
        <div class="col-md-3"><label class="form-label">Average minutes</label><input class="form-control" type="number" min="1" max="600" name="avg_minutes" value="<?= e((string)($record['avg_minutes'] ??
15)) ?>" required></div>
        <div class="col-md-3"><label class="form-label">Daily capacity</label><input class="form-control" type="number" min="1" max="5000" name="daily_capacity" value="<?= e((string)
($record['daily_capacity'] ?? 40)) ?>" required></div>
      </div>
    <?php elseif ($section === 'notifications'): ?>
      <div class="row g-2">
        <?php if ($record): ?><input type="hidden" name="id" value="<?= (int)$record['id'] ?>"><?php endif; ?>
        <div class="col-md-4"><label class="form-label">Patient</label><select class="form-select" name="patient_id" required><?php foreach ($patients as $p): ?><option value="<?= (int)$p['patient_id'] ?>"
<?= (int)($record['patient_id'] ?? 0) === (int)$p['patient_id'] ? 'selected' : '' ?>><?= e($p['full_name']) ?></option><?php endforeach; ?></select></div>
        <div class="col-md-3"><label class="form-label">Queue entry (optional)</label><select class="form-select" name="queue_id"><option value="">None</option><?php foreach ($queueEntries as $q): ?>
<option value="<?= (int)$q['queue_id'] ?>" <?= (int)($record['queue_id'] ?? 0) === (int)$q['queue_id'] ? 'selected' : '' ?>><?= e('#' . $q['queue_number'] . ' (' . $q['queue_date'] . ')') ?></option><?php
endforeach; ?></select></div>
        <div class="col-md-5"><label class="form-label">Type</label><select class="form-select" name="type"><?php foreach (['booking_confirmation', 'queue_update', 'your_turn', 'reminder', 'report_link']
as $type): ?><option value="<?= $type ?>" <?= ($record['type'] ?? '') === $type ? 'selected' : '' ?>><?= e(ucwords(str_replace('_', ' ', $type))) ?></option><?php endforeach; ?></select></div>
      </div>
      <label class="form-label mt-2">Message</label><textarea class="form-control" name="message" maxlength="320" required><?= e($record['message'] ?? '') ?></textarea>
      <small class="text-muted">Creating a notification only records it as queued; it does not send an SMS.</small>
    <?php endif; ?>
    <div class="d-flex gap-2 mt-3"><button class="btn btn-success"><?= $record ? 'Save changes' : 'Create' ?></button><?php if ($record): ?><a class="btn btn-outline-secondary" href="<?= BASE_URL ?
>/staff/manage.php?section=<?= e($section) ?>">Cancel edit</a><?php endif; ?></div>
  </form>
  <?php endif; ?>
</div>

<div class="table-responsive"><table class="table bg-white align-middle">
  <thead><tr>
    <?php
    $headers = [
        'patients' => ['ID', 'Name', 'Phone', 'Email', 'State', 'Actions'],
        'appointments' => ['ID', 'Patient', 'Service', 'Date', 'Time', 'Status', 'Actions'],
        'staff' => ['ID', 'Name', 'Username', 'Role', 'State', 'Actions'],
        'services' => ['ID', 'Service', 'Avg. minutes', 'Capacity', 'State', 'Actions'],
        'queue' => ['ID', 'Patient', 'Service', 'Date', 'Number', 'Status', 'Actions'],
        'notifications' => ['ID', 'Patient', 'Type', 'Message', 'Delivery', 'Created', 'Actions'],
    ];
    foreach ($headers[$section] as $header): ?><th><?= e($header) ?></th><?php endforeach; ?>
  </tr></thead>
  <tbody>
  <?php foreach ($rows as $row): ?>
    <tr>
```

```
      <?php if ($section === 'patients'): ?>
        <td><?= (int)$row['patient_id'] ?></td><td><?= e($row['full_name']) ?></td><td><?= e($row['phone']) ?></td><td><?= e($row['email'] ?? '') ?></td><td><?= $row['is_active'] ? 'Active' : 'Inactive' ?>
</td>
        <td><a class="btn btn-sm btn-outline-success" href="<?= BASE_URL ?>/staff/patient-report.php?id=<?= (int)$row['patient_id'] ?>">Report</a> <a class="btn btn-sm btn-outline-success" href="<?=
BASE_URL ?>/staff/manage.php?section=patients&edit=<?= (int)$row['patient_id'] ?>">Edit</a><?php if ($row['is_active']): ?><form method="post" class="d-inline"><?= csrf_field() ?><input type="hidden"
name="id" value="<?= (int)$row['patient_id'] ?>"><button class="btn btn-sm btn-outline-danger" name="action" value="deactivate">Deactivate</button></form><?php endif; ?></td>
      <?php elseif ($section === 'appointments'): ?>
        <td><?= (int)$row['appointment_id'] ?></td><td><?= e($row['full_name']) ?></td><td><?= e($row['service_name']) ?></td><td><?= e($row['appointment_date']) ?></td><td><?= e(substr($row['slot_time'],
0, 5)) ?></td><td><?= e($row['status']) ?></td>
        <td><?php if ($row['status'] === 'booked'): ?><a class="btn btn-sm btn-outline-success" href="<?= BASE_URL ?>/staff/manage.php?section=appointments&edit=<?= (int)$row['appointment_id'] ?>">Edit</a>
<form method="post" class="d-inline"><?= csrf_field() ?><input type="hidden" name="id" value="<?= (int)$row['appointment_id'] ?>"><button class="btn btn-sm btn-outline-danger" name="action"
value="cancel">Cancel</button></form><?php endif; ?></td>
      <?php elseif ($section === 'staff'): ?>
        <td><?= (int)$row['staff_id'] ?></td><td><?= e($row['full_name']) ?></td><td><?= e($row['username']) ?></td><td><?= e($row['role']) ?></td><td><?= $row['is_active'] ? 'Active' : 'Inactive' ?></td>
        <td><a class="btn btn-sm btn-outline-success" href="<?= BASE_URL ?>/staff/manage.php?section=staff&edit=<?= (int)$row['staff_id'] ?>">Edit</a><?php if ($row['is_active']): ?><form method="post"
class="d-inline"><?= csrf_field() ?><input type="hidden" name="id" value="<?= (int)$row['staff_id'] ?>"><button class="btn btn-sm btn-outline-danger" name="action" value="deactivate">Deactivate</button>
</form><?php endif; ?></td>
      <?php elseif ($section === 'services'): ?>
        <td><?= (int)$row['service_id'] ?></td><td><?= e($row['service_name']) ?></td><td><?= (int)$row['avg_minutes'] ?></td><td><?= (int)$row['daily_capacity'] ?></td><td><?= $row['is_active'] ? 'Active'
: 'Inactive' ?></td>
        <td><a class="btn btn-sm btn-outline-success" href="<?= BASE_URL ?>/staff/manage.php?section=services&edit=<?= (int)$row['service_id'] ?>">Edit</a><?php if ($row['is_active']): ?><form
method="post" class="d-inline"><?= csrf_field() ?><input type="hidden" name="id" value="<?= (int)$row['service_id'] ?>"><button class="btn btn-sm btn-outline-danger" name="action"
value="deactivate">Deactivate</button></form><?php endif; ?></td>
      <?php elseif ($section === 'queue'): ?>
        <td><?= (int)$row['queue_id'] ?></td><td><?= e($row['full_name']) ?></td><td><?= e($row['service_name']) ?></td><td><?= e($row['queue_date']) ?></td><td>#<?= (int)$row['queue_number'] ?></td><td><?
= e($row['status']) ?><?= $row['is_emergency'] ? ' (priority)' : '' ?></td>
        <td><?php if ($row['status'] === 'waiting'): ?><form method="post" class="d-inline"><?= csrf_field() ?><input type="hidden" name="queue_id" value="<?= (int)$row['queue_id'] ?>"><?php if
(!$row['is_emergency']): ?><button class="btn btn-sm btn-outline-danger" name="action" value="priority">Priority</button><?php endif; ?><button class="btn btn-sm btn-outline-secondary" name="action"
value="no_show">No-show</button></form><?php elseif ($row['status'] === 'called'): ?><form method="post" class="d-inline"><?= csrf_field() ?><input type="hidden" name="queue_id" value="<?=
(int)$row['queue_id'] ?>"><button class="btn btn-sm btn-success" name="action" value="served">Served</button><button class="btn btn-sm btn-outline-secondary" name="action" value="no_show">No-show</button>
</form><?php endif; ?></td>
      <?php elseif ($section === 'notifications'): ?>
        <td><?= (int)$row['notification_id'] ?></td><td><?= e($row['full_name']) ?></td><td><?= e($row['type']) ?></td><td><?= e($row['message']) ?></td><td><?= e($row['delivery_status']) ?></td><td><?=
e($row['created_at']) ?></td>
        <td><a class="btn btn-sm btn-outline-success" href="<?= BASE_URL ?>/staff/manage.php?section=notifications&edit=<?= (int)$row['notification_id'] ?>">Edit</a><form method="post" class="d-inline"><?=
csrf_field() ?><input type="hidden" name="id" value="<?= (int)$row['notification_id'] ?>"><button class="btn btn-sm btn-outline-danger" name="action" value="archive">Archive</button></form></td>
      <?php endif; ?>
    </tr>
  <?php endforeach; ?>
  <?php if (!$rows): ?><tr><td colspan="<?= count($headers[$section]) ?>" class="text-center text-muted">No records found.</td></tr><?php endif; ?>
  </tbody>
</table></div>
<?php require __DIR__ . '/../partials/footer.php'; ?>
```

public/staff/patient-report.php

PHP

```
<?php
require __DIR__ . '/../../src/bootstrap.php';

$reportToken = $_GET['token'] ?? $_POST['report_token'] ?? '';
$linkExpiresAt = null;
if (is_string($reportToken) && preg_match('/\A[a-f0-9]{48}\z/', $reportToken)) {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        csrf_check();
    }
    $linkSt = $db->prepare(
        'SELECT patient_id, expires_at FROM patient_report_links WHERE token_hash=? AND expires_at>NOW()'
    );
    $linkSt->execute([hash('sha256', $reportToken)]);
    $reportLink = $linkSt->fetch();
    if (!$reportLink) {
        http_response_code(403);
        exit('This patient report link is invalid or has expired.');
    }
    $patientId = (int)$reportLink['patient_id'];
    $linkExpiresAt = $reportLink['expires_at'];
    if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
        header('Cache-Control: private, no-store');
        header('Referrer-Policy: no-referrer');
        ?>
        <!doctype html>
        <html lang="en">
        <head>
          <meta charset="utf-8">
          <meta name="viewport" content="width=device-width, initial-scale=1">
          <title>Open patient report</title>
          <link href="<?= BASE_URL ?>/styles.css" rel="stylesheet">
        </head>
        <body class="clinic-app">
          <main class="patient-report">
            <h1>Open your patient report</h1>
            <p>This private report link expires <?= e($linkExpiresAt) ?>.</p>
            <form method="post" action="<?= BASE_URL ?>/staff/patient-report.php">
              <?= csrf_field() ?>
              <input type="hidden" name="report_token" value="<?= e($reportToken) ?>">
              <button class="btn btn-success" type="submit">View report</button>
            </form>
          </main>
        </body>
        </html>
        <?php
        exit;
    }
} else {
    require_staff_role($db, ['receptionist', 'nurse', 'admin']);
    $patientId = filter_var($_GET['id'] ?? null, FILTER_VALIDATE_INT);
    if (!$patientId || $patientId < 1) {
        http_response_code(400);
        exit('A valid patient ID is required.');
    }
}
```

```
$patientSt = $db->prepare(
    'SELECT patient_id, full_name, phone, email, location, ghana_card_number, is_active, created_at
     FROM patients WHERE patient_id=?'
);
$patientSt->execute([$patientId]);
$patient = $patientSt->fetch();
if (!$patient) {
    http_response_code(404);
    exit('Patient not found.');
}

$appointmentSt = $db->prepare(
    'SELECT a.appointment_id, a.appointment_date, a.slot_time, a.reason, a.status,
            s.service_name, q.queue_number, q.status AS queue_status
     FROM appointments a
     JOIN services s ON s.service_id=a.service_id
     LEFT JOIN queue_entries q ON q.appointment_id=a.appointment_id
     WHERE a.patient_id=?
     ORDER BY a.appointment_date DESC, a.slot_time DESC, a.appointment_id DESC'
);
$appointmentSt->execute([$patientId]);
$appointments = $appointmentSt->fetchAll();

$title = 'Patient report - ' . $patient['full_name'];
header('Cache-Control: private, no-store');
header('Referrer-Policy: no-referrer');
?>
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title><?= e($title) ?></title>
  <link href="<?= BASE_URL ?>/styles.css" rel="stylesheet">
</head>
<body class="clinic-app patient-report-page">
  <main class="patient-report">
    <div class="report-actions">
      <button class="btn btn-success" type="button" onclick="window.print()">Print / Save as PDF</button>
      <?php if (!$linkExpiresAt): ?><a class="btn btn-outline-secondary" href="<?= BASE_URL ?>/staff/manage.php?section=patients">Back to patients</a><?php endif; ?>
    </div>
    <?php if ($linkExpiresAt): ?><p class="report-link-notice">This private link expires <?= e($linkExpiresAt) ?>.</p><?php endif; ?>

    <header class="report-header">
      <div>
        <p class="report-kicker">Clinic Queue</p>
        <h1>Patient Report</h1>
        <p class="report-subtitle">Patient profile and appointment history</p>
      </div>
      <div class="report-meta">
        <strong>Report date</strong>
        <span><?= e(date('Y-m-d H:i')) ?></span>
        <strong>Patient ID</strong>
        <span><?= (int)$patient['patient_id'] ?></span>
      </div>
    </header>

    <section class="report-section">
      <h2>Patient details</h2>
      <dl class="report-details">
```

```
      <div><dt>Full name</dt><dd><?= e($patient['full_name']) ?></dd></div>
      <div><dt>Phone</dt><dd><?= e($patient['phone']) ?></dd></div>
      <div><dt>Email</dt><dd><?= e($patient['email'] ?: 'Not provided') ?></dd></div>
      <div><dt>Location</dt><dd><?= e($patient['location'] ?: 'Not provided') ?></dd></div>
      <div><dt>Ghana Card Number</dt><dd><?= e($patient['ghana_card_number'] ?: 'Not provided') ?></dd></div>
      <div><dt>Account status</dt><dd><?= (int)$patient['is_active'] ? 'Active' : 'Inactive' ?></dd></div>
      <div><dt>Registered</dt><dd><?= e($patient['created_at']) ?></dd></div>
    </dl>
  </section>

  <section class="report-section">
    <h2>Appointments and queue history</h2>
    <?php if ($appointments): ?>
      <table class="report-table">
        <thead>
          <tr>
            <th>Date</th>
            <th>Time</th>
            <th>Service</th>
            <th>Appointment status</th>
            <th>Queue</th>
            <th>Reason</th>
          </tr>
        </thead>
        <tbody>
          <?php foreach ($appointments as $appointment): ?>
            <tr>
              <td><?= e($appointment['appointment_date']) ?></td>
              <td><?= e(substr($appointment['slot_time'], 0, 5)) ?></td>
              <td><?= e($appointment['service_name']) ?></td>
              <td><?= e(ucwords(str_replace('_', ' ', $appointment['status']))) ?></td>
              <td><?= $appointment['queue_number'] === null ? '—' : '#' . (int)$appointment['queue_number'] . ' (' . e($appointment['queue_status']) . ')' ?></td>
              <td><?= e($appointment['reason'] ?: '—') ?></td>
            </tr>
          <?php endforeach; ?>
        </tbody>
      </table>
    <?php else: ?>
      <p class="report-empty">No appointments recorded.</p>
    <?php endif; ?>
  </section>

  <footer class="report-footer">
    Confidential patient information. Share and store this report securely.
  </footer>
</main>
</body>
</html>
```

sql/schema.sql

SQL

```sql
-- Clinic Appointment Queue System: database schema (MySQL 8 / MariaDB)
CREATE DATABASE IF NOT EXISTS clinic_queue
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE clinic_queue;

-- Patients who book and join the queue
CREATE TABLE patients (
  patient_id    INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  full_name     VARCHAR(100) NOT NULL,
  phone         VARCHAR(15)  NOT NULL UNIQUE,   -- used for SMS, e.g. 0244123456
  email         VARCHAR(120) NULL,
  location      VARCHAR(150) NULL,
  ghana_card_number VARCHAR(15) NULL,
  password_hash VARCHAR(255) NOT NULL,          -- use password_hash() in PHP
  passport_photo VARCHAR(80) NULL,
  is_active     TINYINT(1) NOT NULL DEFAULT 1,
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Receptionists, nurses, admins
CREATE TABLE staff (
  staff_id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  full_name     VARCHAR(100) NOT NULL,
  username      VARCHAR(50)  NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role          ENUM('receptionist','nurse','admin') NOT NULL DEFAULT 'receptionist',
  is_active     TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB;

-- Services offered (General Consultation, Dental, Antenatal, ...)
CREATE TABLE services (
  service_id       INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  service_name     VARCHAR(80) NOT NULL UNIQUE,
  avg_minutes      SMALLINT UNSIGNED NOT NULL DEFAULT 15,  -- used for wait estimate
  daily_capacity   SMALLINT UNSIGNED NOT NULL DEFAULT 40,
  is_active        TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB;

-- A booking made by a patient
CREATE TABLE appointments (
  appointment_id   INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  patient_id       INT UNSIGNED NOT NULL,
  service_id       INT UNSIGNED NOT NULL,
  appointment_date DATE NOT NULL,
  slot_time        TIME NOT NULL,
  reason           VARCHAR(255) NULL,
  status           ENUM('booked','checked_in','cancelled','completed','no_show')
                   NOT NULL DEFAULT 'booked',
  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_appt_patient FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
  CONSTRAINT fk_appt_service FOREIGN KEY (service_id) REFERENCES services(service_id),
  -- a patient cannot book the same slot twice
  UNIQUE KEY uq_patient_slot (patient_id, appointment_date, slot_time),
  INDEX idx_appt_date (appointment_date, service_id, status)
) ENGINE=InnoDB;
```

```
-- Created when a patient checks in; this IS the live queue
CREATE TABLE queue_entries (
  queue_id       INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  appointment_id INT UNSIGNED NOT NULL UNIQUE,
  queue_date     DATE NOT NULL,
  queue_number   SMALLINT UNSIGNED NOT NULL,     -- resets every day
  is_emergency   TINYINT(1) NOT NULL DEFAULT 0,  -- staff can push to the front
  status         ENUM('waiting','called','served','no_show') NOT NULL DEFAULT 'waiting',
  checked_in_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  called_at      DATETIME NULL,
  served_at      DATETIME NULL,
  handled_by     INT UNSIGNED NULL,              -- staff who called the patient
  CONSTRAINT fk_queue_appt  FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id),
  CONSTRAINT fk_queue_staff FOREIGN KEY (handled_by) REFERENCES staff(staff_id),
  UNIQUE KEY uq_daily_number (queue_date, queue_number),
  INDEX idx_queue_live (queue_date, status, is_emergency, queue_number)
) ENGINE=InnoDB;

-- Log of every SMS sent (also useful as test evidence)
CREATE TABLE notifications (
  notification_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  patient_id      INT UNSIGNED NOT NULL,
  queue_id        INT UNSIGNED NULL,
  type            ENUM('booking_confirmation','queue_update','your_turn','reminder','report_link') NOT NULL,
  message         VARCHAR(320) NOT NULL,
  delivery_status ENUM('queued','sent','failed') NOT NULL DEFAULT 'queued',
  provider_ref    VARCHAR(100) NULL,             -- message id from the SMS API
  archived_at     DATETIME NULL,
  sent_at         DATETIME NULL,
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_notif_patient FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
  CONSTRAINT fk_notif_queue   FOREIGN KEY (queue_id)   REFERENCES queue_entries(queue_id)
) ENGINE=InnoDB;

CREATE TABLE patient_report_links (
  report_link_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  patient_id     INT UNSIGNED NOT NULL,
  token_hash     CHAR(64) NOT NULL UNIQUE,
  expires_at     DATETIME NOT NULL,
  created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_report_link_patient FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
  INDEX idx_report_link_expiry (expires_at)
) ENGINE=InnoDB;

-- ---------- Seed data for development and demo ----------
INSERT INTO services (service_name, avg_minutes, daily_capacity) VALUES
  ('General Consultation', 15, 40),
  ('Dental', 25, 15),
  ('Antenatal Care', 20, 20),
  ('Laboratory', 10, 50);

-- Create staff accounts with tools/create_staff.php so passwords are hashed
-- by PHP; do not seed staff with a placeholder password hash.

-- ---------- Queries your code will need ----------
-- Next patient to call (emergencies first, then lowest number):
--   SELECT * FROM queue_entries
--   WHERE queue_date = CURDATE() AND status = 'waiting'
--   ORDER BY is_emergency DESC, queue_number ASC LIMIT 1;
--
```

```
-- Next queue number for today (do this inside a transaction):
--   SELECT COALESCE(MAX(queue_number),0)+1 FROM queue_entries
--   WHERE queue_date = CURDATE() FOR UPDATE;
--
-- Patients ahead of me (for position + wait estimate):
--   SELECT COUNT(*) FROM queue_entries
--   WHERE queue_date = CURDATE() AND status = 'waiting'
--     AND (is_emergency > :mine_emergency
--          OR (is_emergency = :mine_emergency AND queue_number < :mine_number));
--   wait_minutes = patients_ahead * services.avg_minutes
```

src/bootstrap.php

PHP

```
<?php
declare(strict_types=1);
session_start();
date_default_timezone_set('Africa/Accra');

$config = require __DIR__ . '/../config/config.php';
define('BASE_URL', $config['base_url']);

require __DIR__ . '/helpers.php';
spl_autoload_register(function (string $class): void {
    $file = __DIR__ . '/' . $class . '.php';
    if (is_file($file)) require $file;
});

$db    = Database::connect($config['db']);
$sms   = new SmsService($db, $config['sms']);
$queue = new QueueService($db, $sms, $config['public_url']);
```

src/Database.php

PHP

```php
<?php
declare(strict_types=1);

class Database
{
    public static function connect(array $c): PDO
    {
        $dsn = "mysql:host={$c['host']};dbname={$c['name']};charset=utf8mb4";
        return new PDO($dsn, $c['user'], $c['pass'], [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES   => false,
        ]);
    }
}
```

src/helpers.php

PHP

```php
<?php
declare(strict_types=1);

function e(?string $s): string { return htmlspecialchars((string)$s, ENT_QUOTES, 'UTF-8'); }

function csrf_token(): string {
    if (empty($_SESSION['csrf'])) $_SESSION['csrf'] = bin2hex(random_bytes(32));
    return $_SESSION['csrf'];
}
function csrf_field(): string { return '<input type="hidden" name="csrf" value="' . csrf_token() . '">'; }
function csrf_check(): void {
    if (!hash_equals($_SESSION['csrf'] ?? '', $_POST['csrf'] ?? '')) {
        http_response_code(419);
        exit('Invalid form token. Go back and try again.');
    }
}

function redirect(string $path): never { header('Location: ' . BASE_URL . $path); exit; }

function flash(?string $msg = null, string $type = 'info'): ?array {
    if ($msg !== null) { $_SESSION['flash'] = ['msg' => $msg, 'type' => $type]; return null; }
    $f = $_SESSION['flash'] ?? null;
    unset($_SESSION['flash']);
    return $f;
}

function require_patient(): void { if (empty($_SESSION['patient_id'])) redirect('/login.php'); }
function require_staff(): void   { if (empty($_SESSION['staff_id']))   redirect('/staff/login.php'); }

function require_staff_role(PDO $db, array $roles): array
{
    require_staff();
    $st = $db->prepare('SELECT role, is_active FROM staff WHERE staff_id=?');
    $st->execute([(int)$_SESSION['staff_id']]);
    $staff = $st->fetch();

    if (!$staff || !(int)$staff['is_active']) {
        unset($_SESSION['staff_id'], $_SESSION['staff_name']);
        redirect('/staff/login.php');
    }
    if (!in_array($staff['role'], $roles, true)) {
        http_response_code(403);
        exit('You do not have permission to access this page.');
    }

    return $staff;
}
```

src/QueueService.php

PHP

```
<?php
declare(strict_types=1);

/** Core queue logic: check-in, calling next, position and wait estimate. */
class QueueService
{
    public function __construct(private PDO $db, private ?SmsService $sms = null, private string $publicUrl = '') {}

    // ---------- Pure functions (unit-testable without a database) ----------

    /** Estimated wait in minutes = people ahead x average minutes per patient. */
    public static function estimateWait(int $ahead, int $avgMinutes): int
    {
        return max(0, $ahead) * max(0, $avgMinutes);
    }

    /** Emergencies first, then lowest queue number. */
    public static function sortQueue(array $entries): array
    {
        usort($entries, function (array $a, array $b): int {
            return [(int)$b['is_emergency'], (int)$a['queue_number']]
               <=> [(int)$a['is_emergency'], (int)$b['queue_number']];
        });
        return $entries;
    }

    // ---------- Database operations ----------

    /** Patient arrives: create a queue entry with today's next number. */
    public function checkIn(int $appointmentId, int $patientId): int
    {
        $today = date('Y-m-d');
        $this->db->beginTransaction();
        try {
            $st = $this->db->prepare(
                "SELECT * FROM appointments WHERE appointment_id=? AND patient_id=? FOR UPDATE"
            );
            $st->execute([$appointmentId, $patientId]);
            $appt = $st->fetch();

            if (!$appt)                          throw new RuntimeException('Appointment not found.');
            if ($appt['status'] !== 'booked')    throw new RuntimeException('This appointment cannot be checked in.');
            if ($appt['appointment_date'] !== $today)
                throw new RuntimeException('You can only check in on the day of your appointment.');

            $n = $this->db->prepare(
                "SELECT COALESCE(MAX(queue_number),0)+1 FROM queue_entries WHERE queue_date=? FOR UPDATE"
            );
            $n->execute([$today]);
            $number = (int)$n->fetchColumn();

            $ins = $this->db->prepare(
                "INSERT INTO queue_entries (appointment_id, queue_date, queue_number) VALUES (?,?,?)"
            );
            $ins->execute([$appointmentId, $today, $number]);
            $queueId = (int)$this->db->lastInsertId();
```

```
        $this->db->prepare("UPDATE appointments SET status='checked_in' WHERE appointment_id=?")
                 ->execute([$appointmentId]);
        $this->db->commit();
    } catch (Throwable $e) {
        if ($this->db->inTransaction()) $this->db->rollBack();
        throw $e;
    }

    [$ahead, $wait] = $this->positionOf($queueId);
    $this->notify($queueId, 'queue_update',
        "Clinic: you are checked in. Queue no. $number, $ahead ahead of you, about $wait min wait.");
    return $queueId;
}

/** @return array [peopleAhead, estimatedWaitMinutes] */
public function positionOf(int $queueId): array
{
    $st = $this->db->prepare(
        "SELECT q.queue_date, q.queue_number, q.is_emergency, s.avg_minutes
           FROM queue_entries q
           JOIN appointments a ON a.appointment_id = q.appointment_id
           JOIN services s     ON s.service_id = a.service_id
          WHERE q.queue_id=?"
    );
    $st->execute([$queueId]);
    $me = $st->fetch();
    if (!$me) return [0, 0];

    $c = $this->db->prepare(
        "SELECT COUNT(*) FROM queue_entries
          WHERE queue_date=? AND status='waiting' AND queue_id<>?
            AND (is_emergency > ? OR (is_emergency = ? AND queue_number < ?))"
    );
    $c->execute([$me['queue_date'], $queueId, $me['is_emergency'], $me['is_emergency'], $me['queue_number']]);
    $ahead = (int)$c->fetchColumn();
    return [$ahead, self::estimateWait($ahead, (int)$me['avg_minutes'])];
}

/** Staff calls the next waiting patient. Returns the entry or null if nobody is waiting. */
public function callNext(int $staffId): ?array
{
    $this->db->beginTransaction();
    try {
        $st = $this->db->prepare(
            "SELECT * FROM queue_entries
              WHERE queue_date=? AND status='waiting'
              ORDER BY is_emergency DESC, queue_number ASC LIMIT 1 FOR UPDATE"
        );
        $st->execute([date('Y-m-d')]);
        $next = $st->fetch();
        if (!$next) { $this->db->commit(); return null; }

        $this->db->prepare(
            "UPDATE queue_entries SET status='called', called_at=NOW(), handled_by=? WHERE queue_id=?"
        )->execute([$staffId, $next['queue_id']]);
        $this->db->commit();
    } catch (Throwable $e) {
        if ($this->db->inTransaction()) $this->db->rollBack();
        throw $e;
```

```
    }

    $this->notify((int)$next['queue_id'], 'your_turn',
        "Clinic: it is your turn. Queue no. {$next['queue_number']}. Please proceed to the consulting room.");
    $this->notifyUpcoming();
    return $next;
}

public function markServed(int $queueId): bool
{
    if (!$this->finish($queueId, 'served', 'completed')) return false;
    return $this->sendPatientReportLink($queueId);
}

public function markNoShow(int $queueId): void { $this->finish($queueId, 'no_show', 'no_show'); }

public function setEmergency(int $queueId, bool $flag): void
{
    $this->db->prepare("UPDATE queue_entries SET is_emergency=? WHERE queue_id=? AND status='waiting'")
             ->execute([$flag ? 1 : 0, $queueId]);
}

/** Today's active queue (waiting + called) with patient and service details, in calling order. */
public function todayQueue(): array
{
    $st = $this->db->prepare(
        "SELECT q.*, p.full_name, p.phone, s.service_name
           FROM queue_entries q
           JOIN appointments a ON a.appointment_id = q.appointment_id
           JOIN patients p     ON p.patient_id = a.patient_id
           JOIN services s     ON s.service_id = a.service_id
          WHERE q.queue_date=? AND q.status IN ('waiting','called')"
    );
    $st->execute([date('Y-m-d')]);
    $rows = $st->fetchAll();
    $waiting = self::sortQueue(array_filter($rows, fn($r) => $r['status'] === 'waiting'));
    $called  = array_filter($rows, fn($r) => $r['status'] === 'called');
    return ['called' => array_values($called), 'waiting' => array_values($waiting)];
}

// ---------- Internals ----------

private function finish(int $queueId, string $queueStatus, string $apptStatus): bool
{
    $updated = $this->db->prepare(
        "UPDATE queue_entries SET status=?, served_at=NOW() WHERE queue_id=? AND status IN ('waiting','called')"
    );
    $updated->execute([$queueStatus, $queueId]);
    if ($updated->rowCount() === 0) return false;

    $this->db->prepare(
        "UPDATE appointments SET status=? WHERE appointment_id=(SELECT appointment_id FROM queue_entries WHERE queue_id=?)"
    )->execute([$apptStatus, $queueId]);
    return true;
}

private function sendPatientReportLink(int $queueId): bool
{
    if (!$this->sms || $this->publicUrl === '') return false;
```

```
        $st = $this->db->prepare(
            'SELECT p.patient_id, p.phone FROM queue_entries q
             JOIN appointments a ON a.appointment_id=q.appointment_id
             JOIN patients p ON p.patient_id=a.patient_id
             WHERE q.queue_id=?'
        );
        $st->execute([$queueId]);
        $patient = $st->fetch();
        if (!$patient) return false;

        $token = bin2hex(random_bytes(24));
        $expiresAt = (new DateTimeImmutable('+7 days'))->format('Y-m-d H:i:s');
        $this->db->exec('DELETE FROM patient_report_links WHERE expires_at<=NOW()');
        $insert = $this->db->prepare(
            'INSERT INTO patient_report_links (patient_id, token_hash, expires_at) VALUES (?,?,?)'
        );
        $insert->execute([(int)$patient['patient_id'], hash('sha256', $token), $expiresAt]);

        $url = rtrim($this->publicUrl, '/') . '/staff/patient-report.php?token=' . rawurlencode($token);
        $message = "Clinic: your visit report is ready. Open and print/save as PDF within 7 days: $url";
        return $this->sms->send((int)$patient['patient_id'], $patient['phone'], 'report_link', $message, $queueId);
    }

    /** Tell the patient who is now 3rd in line (once). */
    private function notifyUpcoming(): void
    {
        $waiting = $this->todayQueue()['waiting'];
        if (count($waiting) < 3) return;
        $third = $waiting[2];

        $chk = $this->db->prepare("SELECT COUNT(*) FROM notifications WHERE queue_id=? AND type='queue_update' AND message LIKE '%3rd%'");
        $chk->execute([$third['queue_id']]);
        if ((int)$chk->fetchColumn() > 0) return;

        $this->notify((int)$third['queue_id'], 'queue_update',
            "Clinic: you are 3rd in line (queue no. {$third['queue_number']}). Please be ready.");
    }

    private function notify(int $queueId, string $type, string $message): void
    {
        if (!$this->sms) return;
        $st = $this->db->prepare(
            "SELECT p.patient_id, p.phone FROM queue_entries q
               JOIN appointments a ON a.appointment_id = q.appointment_id
               JOIN patients p ON p.patient_id = a.patient_id
              WHERE q.queue_id=?"
        );
        $st->execute([$queueId]);
        if ($p = $st->fetch()) {
            $this->sms->send((int)$p['patient_id'], $p['phone'], $type, $message, $queueId);
        }
    }
}
```

src/SmsService.php

PHP

```php
<?php
declare(strict_types=1);

/** Sends SMS and logs every message in the notifications table. */
class SmsService
{
    public function __construct(private PDO $db, private array $cfg) {}

    /** 0244123456 -> +233244123456 */
    public static function toInternational(string $phone): string
    {
        $phone = preg_replace('/\s+/', '', $phone);
        if (str_starts_with($phone, '+')) return $phone;
        if (str_starts_with($phone, '233')) return '+' . $phone;
        return '+233' . ltrim($phone, '0');
    }

    public function send(int $patientId, string $phone, string $type, string $message, ?int $queueId = null): bool
    {
        $st = $this->db->prepare(
            "INSERT INTO notifications (patient_id, queue_id, type, message) VALUES (?,?,?,?)"
        );
        $st->execute([$patientId, $queueId, $type, $message]);
        $id = (int)$this->db->lastInsertId();

        [$ok, $ref] = ($this->cfg['driver'] === 'africastalking')
            ? $this->viaAfricasTalking(self::toInternational($phone), $message)
            : [true, 'log'];

        $up = $this->db->prepare(
            "UPDATE notifications SET delivery_status=?, provider_ref=?, sent_at=? WHERE notification_id=?"
        );
        $up->execute([$ok ? 'sent' : 'failed', $ref, $ok ? date('Y-m-d H:i:s') : null, $id]);
        return $ok;
    }

    private function viaAfricasTalking(string $to, string $message): array
    {
        $url = ($this->cfg['at_username'] === 'sandbox')
            ? 'https://api.sandbox.africastalking.com/version1/messaging'
            : 'https://api.africastalking.com/version1/messaging';

        $ch = curl_init($url);
        curl_setopt_array($ch, [
            CURLOPT_POST           => true,
            CURLOPT_RETURNTRANSFER => true,
            CURLOPT_TIMEOUT        => 10,
            CURLOPT_HTTPHEADER     => ['Accept: application/json', 'apiKey: ' . $this->cfg['at_api_key']],
            CURLOPT_POSTFIELDS     => http_build_query([
                'username' => $this->cfg['at_username'], 'to' => $to, 'message' => $message,
            ]),
        ]);
        $body = curl_exec($ch);
        $code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
        curl_close($ch);
```

```
        $data = json_decode((string)$body, true);
        $ref  = $data['SMSMessageData']['Recipients'][0]['messageId'] ?? null;
        return [$code >= 200 && $code < 300, $ref];
    }
}
```

tests/QueueServiceTest.php

PHP

```
<?php
declare(strict_types=1);

use PHPUnit\Framework\TestCase;

require_once __DIR__ . '/../src/QueueService.php';
require_once __DIR__ . '/../src/SmsService.php';

final class QueueServiceTest extends TestCase
{
    public function testEstimateWaitMultipliesPeopleAheadByAverage(): void
    {
        $this->assertSame(45, QueueService::estimateWait(3, 15));
    }

    public function testEstimateWaitIsZeroWhenNobodyAhead(): void
    {
        $this->assertSame(0, QueueService::estimateWait(0, 15));
    }

    public function testEstimateWaitNeverNegative(): void
    {
        $this->assertSame(0, QueueService::estimateWait(-2, 15));
    }

    public function testSortQueuePutsEmergencyFirstThenByNumber(): void
    {
        $entries = [
            ['queue_number' => 1, 'is_emergency' => 0],
            ['queue_number' => 2, 'is_emergency' => 0],
            ['queue_number' => 5, 'is_emergency' => 1],
            ['queue_number' => 4, 'is_emergency' => 1],
        ];
        $sorted = array_column(QueueService::sortQueue($entries), 'queue_number');
        $this->assertSame([4, 5, 1, 2], $sorted);
    }

    public function testPhoneConvertedToInternationalFormat(): void
    {
        $this->assertSame('+233244123456', SmsService::toInternational('0244123456'));
        $this->assertSame('+233244123456', SmsService::toInternational('233244123456'));
        $this->assertSame('+233244123456', SmsService::toInternational('+233244123456'));
    }
}
```

tools/create_staff.php

PHP

```
<?php
// Usage: php tools/create_staff.php admin admin123 "Clinic Admin"
require __DIR__ . '/../src/Database.php';
$config = require __DIR__ . '/../config/config.php';
[$script, $user, $pass] = $argv + [null, null, null];
$name = $argv[3] ?? $user;
if (!$user || !$pass) exit("Usage: php tools/create_staff.php <username> <password> [\"Full Name\"]\n");

$db = Database::connect($config['db']);
$db->prepare(
    "INSERT INTO staff (full_name, username, password_hash, role) VALUES (?,?,?, 'admin')
     ON DUPLICATE KEY UPDATE password_hash=VALUES(password_hash), full_name=VALUES(full_name), is_active=1"
)->execute([$name, $user, password_hash($pass, PASSWORD_DEFAULT)]);
echo "Staff account '$user' is ready.\n";
```

storage/.htaccess

TEXT

```
Require all denied
```